```
// ===== File: src/app/_layout.tsx =====
import { useCallback, useState } from 'react';
import { Stack } from 'expo-router';
import { StatusBar } from 'expo-status-bar';

import { AuthProvider } from '@/features/auth';
import { OpeningAnimation } from '@/components/OpeningAnimation';

export default function RootLayout() {
  const [showOpening, setShowOpening] = useState(true);

  const handleFinish = useCallback(() => {
    setShowOpening(false);
  }, []);

  return (
    <AuthProvider>
      <Stack screenOptions={{ headerShown: false }}>
        <Stack.Screen name="index" />
        <Stack.Screen name="(auth)" />
        <Stack.Screen name="(app)" />
      </Stack>
      <StatusBar style="auto" />
      {showOpening && <OpeningAnimation onFinish={handleFinish} />}
    </AuthProvider>
  );
}



// ===== File: src/app/index.tsx =====
import { ActivityIndicator, StyleSheet, View } from 'react-native';
import { Redirect, type Href } from 'expo-router';

import { useAuth } from '@/features/auth';

const COURSES_ROUTE = '/(app)/courses' as Href;
const LOGIN_ROUTE = '/(auth)/login' as Href;

export default function IndexScreen() {
  const { isLoading, isAuthenticated } = useAuth();

  if (isLoading) {
    return (
      <View style={styles.loading}>
        <ActivityIndicator size="large" />
      </View>
    );
  }

  if (isAuthenticated) {
    return <Redirect href={COURSES_ROUTE} />;
  }

  return <Redirect href={LOGIN_ROUTE} />;
}

const styles = StyleSheet.create({
  loading: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

```
// ===== File: src/app/(auth)/_layout.tsx =====
import { ActivityIndicator, StyleSheet, View } from 'react-native';
import { Redirect, Stack, type Href } from 'expo-router';

import { useAuth } from '@/features/auth';

const COURSES_ROUTE = '/(app)/courses' as Href;

export default function AuthLayout() {
  const { isLoading, isAuthenticated } = useAuth();

  if (isLoading) {
    return (
      <View style={styles.loading}>
        <ActivityIndicator size="large" />
      </View>
    );
  }

  if (isAuthenticated) {
    return <Redirect href={COURSES_ROUTE} />;
  }

  return <Stack screenOptions={{ headerShown: false }} />;
}

const styles = StyleSheet.create({
  loading: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
});



// ===== File: src/app/(auth)/login.tsx =====
import { LoginScreen } from '@/features/auth/components/LoginScreen';

export default LoginScreen;



// ===== File: src/app/(app)/_layout.tsx =====
import { ActivityIndicator, StyleSheet, View } from 'react-native';
import { Redirect, Stack, type Href } from 'expo-router';

import { useAuth } from '@/features/auth';

const LOGIN_ROUTE = '/(auth)/login' as Href;

export default function AppLayout() {
  const { isLoading, isAuthenticated } = useAuth();

  if (isLoading) {
    return (
      <View style={styles.loading}>
        <ActivityIndicator size="large" />
      </View>
    );
  }
```

```
  if (!isAuthenticated) {
    return <Redirect href={LOGIN_ROUTE} />;
  }

  return <Stack screenOptions={{ headerShown: false }} />;
}

const styles = StyleSheet.create({
  loading: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

```
// ===== File: src/app/(app)/courses/index.tsx =====
import { CourseHomeScreen } from '@/features/courses/components/CourseHomeScreen';

export default function CoursesRoute() {
  return <CourseHomeScreen />;
}
```

```
// ===== File: src/app/(app)/lesson.tsx =====
import { LessonScreen } from '@/features/lesson/LessonScreen';

export default function LessonRoute() {
  return <LessonScreen />;
}
```

```
// ===== File: src/features/auth/AuthProvider.tsx =====
import {
  createContext,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useRef,
  useState,
  type PropsWithChildren,
} from 'react';
import * as SplashScreen from 'expo-splash-screen';

import { createApiClient, type ApiClient } from '@/lib/api/client';
import { clearFtAuth, getFtAuth, setFtAuth } from '@/lib/auth/storage';
import type { FtAuthRecord } from '@/lib/auth/types';
import { getApiHost } from '@/lib/env';
import { checkSession } from '@/features/auth/services/login';

SplashScreen.preventAutoHideAsync().catch(() => {
  // Splash may already be hidden during fast refresh.
});

type AuthContextValue = {
  auth: FtAuthRecord | null;
  isLoading: boolean;
  isAuthenticated: boolean;
  apiClient: ApiClient;
```

```
  refreshAuth: () => Promise<FtAuthRecord | null>;
  saveAuth: (auth: FtAuthRecord) => Promise<void>;
  logout: () => Promise<void>;
};

const AuthContext = createContext<AuthContextValue | null>(null);

export function AuthProvider({ children }: PropsWithChildren) {
  const [auth, setAuthState] = useState<FtAuthRecord | null>(null);
  const [isLoading, setIsLoading] = useState(true);
  const authRef = useRef<FtAuthRecord | null>(null);
  const logoutRef = useRef<() => Promise<void>>(async () => {});

  authRef.current = auth;

  const apiClient = useMemo(
    () =>
      createApiClient({
        baseUrl: getApiHost(),
        getAccessToken: () => authRef.current?.token ?? null,
        onUnauthorized: () => {
          void logoutRef.current();
        },
      }),
    [],
  );

  const refreshAuth = useCallback(async () => {
    const nextAuth = await getFtAuth();
    setAuthState(nextAuth);
    authRef.current = nextAuth;
    return nextAuth;
  }, []);

  const saveAuth = useCallback(async (nextAuth: FtAuthRecord) => {
    await setFtAuth(nextAuth);
    setAuthState(nextAuth);
    authRef.current = nextAuth;
  }, []);

  const logout = useCallback(async () => {
    await clearFtAuth();
    setAuthState(null);
    authRef.current = null;
  }, []);

  logoutRef.current = logout;

  useEffect(() => {
    let cancelled = false;

    void (async () => {
      try {
        const stored = await getFtAuth();
        if (!stored?.token) {
          if (stored) {
            await clearFtAuth();
          }
          if (!cancelled) {
            setAuthState(null);
            authRef.current = null;
          }
          return;
        }
```

```
        authRef.current = stored;
        if (!cancelled) {
          setAuthState(stored);
        }

        try {
          const verified = await checkSession(apiClient, stored);
          await setFtAuth(verified);
          if (!cancelled) {
            setAuthState(verified);
            authRef.current = verified;
          }
        } catch {
          await clearFtAuth();
          if (!cancelled) {
            setAuthState(null);
            authRef.current = null;
          }
        }
      } finally {
        if (!cancelled) {
          setIsLoading(false);
          await SplashScreen.hideAsync();
        }
      }
    })();

    return () => {
      cancelled = true;
    };
  }, [apiClient]);

  const value = useMemo<AuthContextValue>(
    () => ({
      auth,
      isLoading,
      isAuthenticated: Boolean(auth?.token),
      apiClient,
      refreshAuth,
      saveAuth,
      logout,
    }),
    [apiClient, auth, isLoading, logout, refreshAuth, saveAuth],
  );

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth(): AuthContextValue {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
}



// ===== File: src/features/auth/components/LoginScreen.tsx =====
import { useCallback, useEffect, useRef, useState } from 'react';
import {
  KeyboardAvoidingView,
  Platform,
```

```
  ScrollView,
  StyleSheet,
  useWindowDimensions,
  View,
} from 'react-native';
import { Image } from 'expo-image';
import { useRouter, type Href } from 'expo-router';

import { ApiRequestError } from '@/lib/api/client';
import { readRememberMePreference, writeRememberMePreference } from '@/lib/auth/preferences';
import { getApiHost } from '@/lib/env';

import { useAuth } from '../AuthProvider';
import { loginImages } from '../assets/loginAssets';
import { useWechatQrLogin } from '../hooks/useWechatQrLogin';
import type { FtAuthRecord } from '@/lib/auth/types';
import {
  LoginError,
  loginWithFrontpage,
  loginWithWechatCode,
  checkSession,
} from '../services/login';
import { requestWechatAuthCode } from '../services/wechatNative';
import { LoginColors } from './LoginConstants';
import { LandingView } from './LandingView';
import { LoginView } from './LoginView';
import { WechatModal } from './WechatModal';

type ViewMode = 'landing' | 'login';
type LoginTab = 'personal' | 'school';

const COURSES_ROUTE = '/(app)/courses' as Href;

export function LoginScreen() {
  const router = useRouter();
  const { apiClient, saveAuth } = useAuth();
  const { height: windowHeight, width: windowWidth } = useWindowDimensions();
  const shortestSide = Math.min(windowWidth, windowHeight);

  // Treat only true tablet/desktop layouts as desktop so landscape phones keep the compact m
  const isDesktopLayout = shortestSide >= 760;
  const logoHeight = isDesktopLayout
    ? Math.min(56, Math.max(36, windowWidth * 0.04))
    : 34;
  const logoLeft = isDesktopLayout ? 28 : 18;
  const logoTop = isDesktopLayout ? 56 - logoHeight / 2 : 40;

  const [viewMode, setViewMode] = useState<ViewMode>('landing');
  const [activeTab, setActiveTab] = useState<LoginTab>('personal');
  const [rememberMe, setRememberMe] = useState(true);
  const [statusMessage, setStatusMessage] = useState('');
  const [errorMessage, setErrorMessage] = useState('');
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [wechatModalVisible, setWechatModalVisible] = useState(false);

  // Keep rememberMe in a ref so finishLogin closures don't go stale
  const rememberMeRef = useRef(rememberMe);
  rememberMeRef.current = rememberMe;

  useEffect(() => {
    void readRememberMePreference().then(setRememberMe);
  }, []);

  const handleRememberMeChange = useCallback(async (value: boolean) => {
```

```
  setRememberMe(value);
  await writeRememberMePreference(value);
}, []);

/* ---- Shared login finish handler ---- */
const finishLogin = useCallback(
  async (action: () => Promise<void>) => {
    setIsSubmitting(true);
    setErrorMessage('');
    setStatusMessage('正在登录...');

    try {
      await action();
      setStatusMessage('登录成功，正在进入...');
      router.replace(COURSES_ROUTE);
    } catch (error) {
      const message =
        error instanceof LoginError ||
        error instanceof ApiRequestError ||
        error instanceof Error
          ? error.message
          : '登录失败，请稍后重试';
      if (error instanceof ApiRequestError) {
        setStatusMessage(`当前 API：${getApiHost()}`);
      } else {
        setStatusMessage('');
      }
      setErrorMessage(message);
    } finally {
      setIsSubmitting(false);
    }
  },
  [router],
);

/* ---- Landing View Handlers ---- */
const handleReturningUser = useCallback(async () => {
  try {
    await checkSession(apiClient, null);
    router.replace(COURSES_ROUTE);
  } catch {
    setViewMode('login');
    setErrorMessage('登录已过期，请重新登录');
  }
}, [apiClient, router]);

const handleNewUser = useCallback(() => {
  setActiveTab('personal');
  setViewMode('login');
  setErrorMessage('');
  setStatusMessage('');
}, []);

/* ---- Login View Handlers ---- */
const handleHomePress = useCallback(() => {
  setViewMode('landing');
  setErrorMessage('');
  setStatusMessage('');
}, []);

const handlePersonalSubmit = useCallback(
  (phone: string, password: string) => {
    const trimmedPhone = phone.trim();
    const trimmedPassword = password.trim();
```

```
    if (!trimmedPhone || !trimmedPassword) {
      setErrorMessage('请输入手机号和密码');
      return;
    }

    void finishLogin(async () => {
      const auth = await loginWithFrontpage(apiClient, {
        mode: 'personal',
        rememberMe: rememberMeRef.current,
        payload: {
          phone: trimmedPhone,
          name: trimmedPassword,
        },
      });
      await saveAuth(auth);
    });
  },
  [apiClient, saveAuth, finishLogin],
);

const handleSchoolSubmit = useCallback(
  (school: string, className: string, studentName: string, schoolCode: string) => {
    const s = school.trim();
    const c = className.trim();
    const n = studentName.trim();
    const code = schoolCode.trim();
    if (!s || !c || !n || !code) {
      setErrorMessage('请完整填写学校、班级、姓名和密码');
      return;
    }

    void finishLogin(async () => {
      const auth = await loginWithFrontpage(apiClient, {
        mode: 'school',
        rememberMe: rememberMeRef.current,
        payload: {
          school: s,
          class_name: c,
          student_name: n,
          school_code: code,
        },
      });
      await saveAuth(auth);
    });
  },
  [apiClient, saveAuth, finishLogin],
);

const handleWechatLoginPress = useCallback(() => {
  setWechatModalVisible(true);
}, []);

const handleWechatLogin = useCallback(() => {
  setWechatModalVisible(false);

  void finishLogin(async () => {
    const code = await requestWechatAuthCode();
    const auth = await loginWithWechatCode(apiClient, code, rememberMeRef.current);
    await saveAuth(auth);
  });
}, [apiClient, saveAuth, finishLogin]);

/* ---- QR scan login callbacks ---- */
const handleQrLoginSuccess = useCallback(
```

```
  async (auth: FtAuthRecord) => {
    await saveAuth(auth);
    setStatusMessage('扫码登录成功，正在进入...');
    setErrorMessage('');
    router.replace(COURSES_ROUTE);
  },
  [saveAuth, router],
);

const handleQrLoginError = useCallback((error: string) => {
  setErrorMessage(error);
}, []);

const qrLogin = useWechatQrLogin(apiClient, {
  rememberMe,
  onLoginSuccess: handleQrLoginSuccess,
  onLoginError: handleQrLoginError,
});

return (
  <KeyboardAvoidingView
    style={styles.root}
    behavior={Platform.OS === 'ios' ? 'padding' : undefined}>
    <View style={styles.page}>
      {/* Background image layer */}
      <Image
        source={loginImages.background}
        style={styles.backgroundImage}
        contentFit="cover"
      />

      {/* Scrollable scene (matches web mobile: page overflow:auto, scene position:relative
      <ScrollView
        style={styles.sceneScroll}
        contentContainerStyle={styles.sceneContent}
        bounces={false}
        keyboardShouldPersistTaps="handled">
        <View style={[styles.scene, { minHeight: windowHeight }]}>
          {/* Logo — absolute within scene, matches web .brand */}
          <Image
            source={loginImages.logo}
            style={[
              styles.logoBase,
              {
                left: logoLeft,
                top: logoTop,
                height: logoHeight,
                // Estimate width from logo aspect ratio (~3.2:1)
                width: logoHeight * 3.2,
              },
            ]}
            contentFit="contain"
          />

          {viewMode === 'landing' ? (
            <LandingView
              onReturningUser={handleReturningUser}
              onNewUser={handleNewUser}
              windowWidth={windowWidth}
              windowHeight={windowHeight}
              qrLogin={qrLogin}
            />
          ) : (
            <LoginView
```

```
                activeTab={activeTab}
                onTabChange={setActiveTab}
                onHomePress={handleHomePress}
                isDesktopLayout={isDesktopLayout}
                isSubmitting={isSubmitting}
                statusMessage={statusMessage}
                errorMessage={errorMessage}
                rememberMe={rememberMe}
                onRememberMeChange={handleRememberMeChange}
                onWechatLoginPress={handleWechatLoginPress}
                onPersonalSubmit={handlePersonalSubmit}
                onSchoolSubmit={handleSchoolSubmit}
              />
            )}
          </View>
        </ScrollView>
      </View>

      <WechatModal
        visible={wechatModalVisible}
        isSubmitting={isSubmitting}
        onClose={() => setWechatModalVisible(false)}
        onWechatLogin={handleWechatLogin}
      />
    </KeyboardAvoidingView>
  );
}

const styles = StyleSheet.create({
  root: {
    flex: 1,
    backgroundColor: LoginColors.skyBg,
  },
  page: {
    flex: 1,
    backgroundColor: LoginColors.skyBg,
  },
  backgroundImage: {
    ...StyleSheet.absoluteFill,
    width: '100%',
    height: '100%',
  },
  sceneScroll: {
    flex: 1,
  },
  sceneContent: {
    flexGrow: 1,
  },
  scene: {
    position: 'relative',
    width: '100%',
  },
  logoBase: {
    position: 'absolute',
    zIndex: 5,
  },
});



// ===== File: src/features/auth/components/LoginView.tsx =====
import { Pressable, StyleSheet, Text, View } from 'react-native';

import { LoginColors, LoginSizes, LoginWeights } from './LoginConstants';
```

```
import { PersonalForm } from './PersonalForm';
import { SchoolForm } from './SchoolForm';

type LoginTab = 'personal' | 'school';

type LoginViewProps = {
  activeTab: LoginTab;
  onTabChange: (tab: LoginTab) => void;
  onHomePress: () => void;
  isDesktopLayout: boolean;
  isSubmitting: boolean;
  statusMessage: string;
  errorMessage: string;
  rememberMe: boolean;
  onRememberMeChange: (value: boolean) => void;
  onWechatLoginPress: () => void;
  onPersonalSubmit: (phone: string, password: string) => void;
  onSchoolSubmit: (school: string, className: string, studentName: string, schoolCode: string
};

export function LoginView({
  activeTab,
  onTabChange,
  onHomePress,
  isDesktopLayout,
  isSubmitting,
  statusMessage,
  errorMessage,
  rememberMe,
  onRememberMeChange,
  onWechatLoginPress,
  onPersonalSubmit,
  onSchoolSubmit,
}: LoginViewProps) {
  return (
    <View style={styles.container}>
      {/* Home Button — absolute within the scene, matches web .home-button */}
      <Pressable style={styles.homeBtn} onPress={onHomePress} disabled={isSubmitting}>
        <View style={styles.homeIcon}>
          <View style={styles.homeIconRoof} />
          <View style={styles.homeIconBody} />
        </View>
        <Text style={styles.homeBtnText}>返回首页</Text>
      </Pressable>

      {/* Login Stage — matches web .login-stage: padding 86px 16px 32px */}
      <View style={styles.loginStage}>
        {/* Login Panel — matches web .login-panel */}
        <View style={[styles.panel, isDesktopLayout && styles.panelDesktop]}>
          {/* Tabs */}
          <View style={styles.tabs}>
            <Pressable
              style={styles.tab}
              onPress={() => onTabChange('personal')}
              disabled={isSubmitting}>
              <Text style={[styles.tabText, activeTab === 'personal' && styles.tabTextActive]
                个人登录
              </Text>
              {activeTab === 'personal' && <View style={styles.tabIndicator} />}
            </Pressable>
            <Pressable
              style={styles.tab}
              onPress={() => onTabChange('school')}
              disabled={isSubmitting}>
```

```
              <Text style={[styles.tabText, activeTab === 'school' && styles.tabTextActive]}>
                学校登录
              </Text>
              {activeTab === 'school' && <View style={styles.tabIndicator} />}
            </Pressable>
          </View>

          {/* Forms */}
          <View style={styles.forms}>
            {activeTab === 'personal' ? (
              <PersonalForm
                isSubmitting={isSubmitting}
                rememberMe={rememberMe}
                onRememberMeChange={onRememberMeChange}
                onWechatLoginPress={onWechatLoginPress}
                onSubmit={onPersonalSubmit}
              />
            ) : (
              <SchoolForm isSubmitting={isSubmitting} onSubmit={onSchoolSubmit} />
            )}

            {/* Status / Error messages */}
            {statusMessage ? (
              <Text style={[styles.note, { color: LoginColors.success }]}>{statusMessage}</Te
            ) : null}
            {errorMessage ? (
              <Text style={[styles.note, { color: LoginColors.error }]}>{errorMessage}</Text>
            ) : null}
          </View>

          {/* Agreement footer */}
          <View style={styles.agreement}>
            <Text style={styles.agreementText}>
              登录即表示同意
              <Text style={styles.agreementLink}>《用户协议》</Text>
              <Text style={styles.agreementLink}>《隐私政策》</Text>
              <Text style={styles.agreementLink}>《儿童隐私保护声明》</Text>
            </Text>
          </View>
        </View>
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    position: 'relative',
    width: '100%',
  },

  /* ---- Home Button — matches web .home-button (mobile: top:40, right:14, h:44) ---- */
  homeBtn: {
    position: 'absolute',
    top: LoginSizes.homeBtnTop,
    right: LoginSizes.homeBtnRight,
    zIndex: 5,
    flexDirection: 'row',
    alignItems: 'center',
    gap: 8,
    height: LoginSizes.homeBtnHeight,
    paddingHorizontal: 15,
    borderRadius: 999,
    backgroundColor: LoginColors.homeBtnBg,
```

```
},
homeIcon: {
  width: LoginSizes.homeIconW,
  height: LoginSizes.homeIconH,
  justifyContent: 'flex-end',
  alignItems: 'center',
},
homeIconRoof: {
  width: 0,
  height: 0,
  borderLeftWidth: LoginSizes.homeIconW / 2,
  borderRightWidth: LoginSizes.homeIconW / 2,
  borderBottomWidth: 6,
  borderLeftColor: 'transparent',
  borderRightColor: 'transparent',
  borderBottomColor: LoginColors.orange,
},
homeIconBody: {
  width: LoginSizes.homeIconW - 4,
  height: 11,
  backgroundColor: LoginColors.orange,
  borderTopLeftRadius: 1,
  borderTopRightRadius: 1,
},
homeBtnText: {
  fontSize: LoginSizes.homeBtnFontSize,
  fontWeight: LoginWeights.black,
  color: LoginColors.homeBtnText,
},

/* ---- Login Stage — matches web .login-stage (mobile) ---- */
loginStage: {
  width: '100%',
  alignItems: 'center',
  paddingTop: LoginSizes.loginStagePaddingTop,
  paddingHorizontal: LoginSizes.loginStagePaddingH,
  paddingBottom: LoginSizes.loginStagePaddingBottom,
},

/* ---- Panel — matches web .login-panel ---- */
panel: {
  width: '100%',
  maxWidth: LoginSizes.panelMaxWidth,
  borderRadius: LoginSizes.panelBorderRadius,
  backgroundColor: LoginColors.panelBg,
  overflow: 'hidden',
  shadowColor: LoginColors.shadowColor,
  shadowOffset: { width: 0, height: 20 },
  shadowOpacity: 0.1,
  shadowRadius: 44,
  elevation: 10,
},
panelDesktop: {
  aspectRatio: LoginSizes.panelAspectW / LoginSizes.panelAspectH,
},

/* ---- Tabs — matches web .tabs ---- */
tabs: {
  flexDirection: 'row',
  height: LoginSizes.tabHeight,
  borderBottomWidth: LoginSizes.tabBorderBottom,
  borderBottomColor: LoginColors.line,
},
tab: {
```

```
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    position: 'relative',
  },
  tabText: {
    fontSize: LoginSizes.tabFontSize,
    fontWeight: LoginWeights.extraBlack,
    color: LoginColors.tabInactive,
  },
  tabTextActive: {
    color: LoginColors.orange,
  },
  tabIndicator: {
    position: 'absolute',
    left: LoginSizes.tabIndicatorLeft,
    right: LoginSizes.tabIndicatorRight,
    bottom: -LoginSizes.tabBorderBottom,
    height: LoginSizes.tabActiveIndicatorHeight,
    borderTopLeftRadius: LoginSizes.tabActiveIndicatorHeight,
    borderTopRightRadius: LoginSizes.tabActiveIndicatorHeight,
    backgroundColor: LoginColors.orange,
  },

  /* ---- Forms — matches web .forms ---- */
  forms: {
    marginTop: 10,
    paddingTop: LoginSizes.formPaddingTop,
    paddingHorizontal: LoginSizes.formPaddingHorizontal,
    paddingBottom: 4,
  },
  note: {
    marginTop: 10,
    fontSize: LoginSizes.noteFontSize,
    fontWeight: LoginWeights.extraBold,
    textAlign: 'center',
    minHeight: 20,
  },

  /* ---- Agreement — matches web .agreement ---- */
  agreement: {
    paddingHorizontal: 16,
    paddingTop: 14,
    paddingBottom: 14,
  },
  agreementText: {
    fontSize: LoginSizes.agreementFontSize,
    fontWeight: LoginWeights.extraBold,
    color: LoginColors.agreement,
    textAlign: 'center',
    lineHeight: LoginSizes.agreementFontSize * 1.45,
  },
  agreementLink: {
    color: LoginColors.agreementLink,
  },
});



// ===== File: src/features/auth/services/login.ts =====
import type { ApiClient } from '@/lib/api/client';
import { withAccessToken } from '@/lib/api/client';
import {
  buildFtAuthFromCheckResponse,
```

```
  buildFtAuthFromLoginResponse,
  type LoginSuccessResponse,
  type CheckSessionResponse,
} from '@/lib/auth/session';
import type { FtAuthRecord } from '@/lib/auth/types';

export class LoginError extends Error {
  constructor(message: string) {
    super(message);
    this.name = 'LoginError';
  }
}

async function parseJson<T>(response: Response): Promise<T> {
  try {
    return (await response.json()) as T;
  } catch {
    return {} as T;
  }
}

function getErrorMessage(payload: { detail?: string; message?: string }, fallback: string) {
  return payload.detail || payload.message || fallback;
}

export async function checkSession(
  apiClient: ApiClient,
  previous?: FtAuthRecord | null,
): Promise<FtAuthRecord> {
  const response = await apiClient.get('/api/v1/check');
  if (!response.ok) {
    throw new LoginError('登录已失效，请重新登录');
  }

  const payload = await parseJson<CheckSessionResponse>(response);
  return buildFtAuthFromCheckResponse(payload, previous);
}

export async function loginWithFrontpage(
  apiClient: ApiClient,
  options: {
    mode: 'personal' | 'school';
    payload: Record<string, string>;
    rememberMe: boolean;
  },
): Promise<FtAuthRecord> {
  const params = new URLSearchParams({
    remember_me: options.rememberMe ? 'true' : 'false',
  });

  const response = await apiClient.post(
    `/login/v1/frontpage_login?${params.toString()}`,
    {
      mode: options.mode,
      ...options.payload,
    },
  );

  const data = await parseJson<LoginSuccessResponse & { detail?: string; message?: string }>(
    response,
  );

  if (!response.ok || !data.success) {
    throw new LoginError(getErrorMessage(data, '登录失败，请检查填写内容'));
```

```
  }

  return buildFtAuthFromLoginResponse(data, options.rememberMe);
}

export async function registerWechatUser(
  apiClient: ApiClient,
  openid: string,
): Promise<{ name?: string; phone?: string; logo?: string } | null> {
  const response = await apiClient.post('/api/v1/register_user', {
    wechat_openid: openid,
    name: '',
    phone: '',
  });

  if (!response.ok) {
    return null;
  }

  const payload = await parseJson<{ user?: { name?: string; phone?: string; logo?: string } }
    response,
  );
  return payload.user ?? null;
}

export async function loginWithWechatCode(
  apiClient: ApiClient,
  code: string,
  rememberMe: boolean,
): Promise<FtAuthRecord> {
  const response = await apiClient.post('/login/v1/wechat/exchange_code', { code });
  const data = await parseJson<LoginSuccessResponse & { detail?: string; message?: string }>(
    response,
  );

  if (!response.ok || !data.token) {
    throw new LoginError(getErrorMessage(data, '微信登录失败，请重试'));
  }

  const authedClient = withAccessToken(apiClient, data.token);

  let auth = buildFtAuthFromLoginResponse(
    {
      token: data.token,
      expires_in: data.expires_in,
      user_info: data.user_info,
    },
    rememberMe,
  );

  if (auth.userId) {
    const registeredUser = await registerWechatUser(authedClient, auth.userId);
    if (registeredUser) {
      const username = registeredUser.name || auth.username || '';
      auth = {
        ...auth,
        username,
        name: username,
        phone: registeredUser.phone || auth.phone,
        logo: registeredUser.logo || auth.logo,
        hasUsername: Boolean(username),
      };
    }
  }
```

```
  try {
    auth = await checkSession(authedClient, auth);
  } catch {
    // exchange_code already issued a token; keep mapped auth if check fails transiently.
  }

  return auth;
}

/* ================================================================
 *  WeChat QR Code Scan Login (web OAuth via /wechat/qrcode + /wechat/poll)
 * ================================================================ */

export type QrCodeData = {
  qr_url: string;
  state: string;
  expires_in: number;
};

export type QrPollStatus = 'waiting' | 'scanned' | 'confirmed' | 'expired' | 'success';

export type QrPollResult = {
  status: QrPollStatus;
  token?: string;
  expires_in?: number;
  user_info?: { openid?: string; username?: string; avatar?: string };
};

export async function fetchWechatQrCode(apiClient: ApiClient): Promise<QrCodeData> {
  const response = await apiClient.get('/login/v1/wechat/qrcode');
  const data = await parseJson<QrCodeData & { detail?: string }>(response);

  if (!response.ok || !data.qr_url || !data.state) {
    throw new LoginError(data.detail || '获取微信二维码失败');
  }

  return data;
}

export async function pollWechatQrLogin(
  apiClient: ApiClient,
  state: string,
): Promise<QrPollResult> {
  const response = await apiClient.get(`/login/v1/wechat/poll?state=${encodeURIComponent(stat
  return parseJson<QrPollResult>(response);
}

export async function loginWithQrPollResult(
  apiClient: ApiClient,
  pollResult: QrPollResult,
  rememberMe: boolean,
): Promise<FtAuthRecord> {
  if (pollResult.status !== 'success' || !pollResult.token) {
    throw new LoginError('扫码登录未完成');
  }

  const authedClient = withAccessToken(apiClient, pollResult.token);

  let auth = buildFtAuthFromLoginResponse(
    {
      token: pollResult.token,
      expires_in: pollResult.expires_in,
      user_info: pollResult.user_info
```

```
        ? {
            openid: pollResult.user_info.openid,
            username: pollResult.user_info.username,
            avatar: pollResult.user_info.avatar,
          }
        : undefined,
    },
    rememberMe,
  );

  if (auth.userId) {
    const registeredUser = await registerWechatUser(authedClient, auth.userId);
    if (registeredUser) {
      const username = registeredUser.name || auth.username || '';
      auth = {
        ...auth,
        username,
        name: username,
        phone: registeredUser.phone || auth.phone,
        logo: registeredUser.logo || auth.logo,
        hasUsername: Boolean(username),
      };
    }
  }

  try {
    auth = await checkSession(authedClient, auth);
  } catch {
    // keep mapped auth if check fails transiently.
  }

  return auth;
}



// ===== File: src/features/auth/hooks/useWechatQrLogin.ts =====
import { useCallback, useEffect, useRef, useState } from 'react';

import type { ApiClient } from '@/lib/api/client';

import {
  fetchWechatQrCode,
  pollWechatQrLogin,
  loginWithQrPollResult,
  type QrPollStatus,
  type QrCodeData,
} from '../services/login';
import type { FtAuthRecord } from '@/lib/auth/types';

export type QrLoginState = {
  /** The URL to render as QR code (WeChat Open Platform OAuth URL) */
  qrUrl: string | null;
  /** Current QR scan status */
  status: 'loading' | QrPollStatus;
  /** Remaining seconds before QR expires */
  expiresIn: number;
};

const POLL_INTERVAL_MS = 2_000;

export function useWechatQrLogin(
  apiClient: ApiClient,
  options: {
```

```
    rememberMe: boolean;
    onLoginSuccess: (auth: FtAuthRecord) => void;
    onLoginError: (error: string) => void;
  },
) {
  const [state, setState] = useState<QrLoginState>({
    qrUrl: null,
    status: 'loading',
    expiresIn: 0,
  });

  const pollTimerRef = useRef<ReturnType<typeof setInterval> | null>(null);
  const countdownTimerRef = useRef<ReturnType<typeof setInterval> | null>(null);
  const mountedRef = useRef(true);
  const stateRef = useRef(state.status);
  stateRef.current = state.status;

  const stopPolling = useCallback(() => {
    if (pollTimerRef.current) {
      clearInterval(pollTimerRef.current);
      pollTimerRef.current = null;
    }
  }, []);

  const stopCountdown = useCallback(() => {
    if (countdownTimerRef.current) {
      clearInterval(countdownTimerRef.current);
      countdownTimerRef.current = null;
    }
  }, []);

  const stopAll = useCallback(() => {
    stopPolling();
    stopCountdown();
  }, [stopPolling, stopCountdown]);

  const startCountdown = useCallback(
    (seconds: number) => {
      stopCountdown();
      let remaining = seconds;
      countdownTimerRef.current = setInterval(() => {
        remaining -= 1;
        if (!mountedRef.current) return;
        setState((prev) => ({ ...prev, expiresIn: Math.max(0, remaining) }));
        if (remaining <= 0) {
          stopCountdown();
          if (stateRef.current === 'waiting') {
            stopPolling();
            setState((prev) => ({ ...prev, status: 'expired' }));
          }
        }
      }, 1_000);
    },
    [stopCountdown, stopPolling],
  );

  const startPolling = useCallback(
    (qrState: string) => {
      stopPolling();
      pollTimerRef.current = setInterval(async () => {
        if (!mountedRef.current) return;
        try {
          const result = await pollWechatQrLogin(apiClient, qrState);
          if (!mountedRef.current) return;
```

```
        if (result.status === 'scanned') {
          setState((prev) => ({ ...prev, status: 'scanned' }));
        } else if (result.status === 'confirmed') {
          setState((prev) => ({ ...prev, status: 'confirmed' }));
        } else if (result.status === 'success') {
          stopAll();
          setState((prev) => ({ ...prev, status: 'success' }));
          // Process the login result
          try {
            const auth = await loginWithQrPollResult(apiClient, result, options.rememberMe)
            if (mountedRef.current) {
              options.onLoginSuccess(auth);
            }
          } catch (err) {
            if (mountedRef.current) {
              options.onLoginError(
                err instanceof Error ? err.message : '扫码登录失败，请重试',
              );
            }
          }
        } else if (result.status === 'expired') {
          stopAll();
          setState((prev) => ({ ...prev, status: 'expired' }));
        }
      } catch {
        // Transient poll error — keep polling
      }
    }, POLL_INTERVAL_MS);
  },
  [apiClient, options, stopAll, stopPolling],
);

const loadQrCode = useCallback(async () => {
  stopAll();
  setState({ qrUrl: null, status: 'loading', expiresIn: 0 });

  try {
    const data: QrCodeData = await fetchWechatQrCode(apiClient);
    if (!mountedRef.current) return;
    setState({ qrUrl: data.qr_url, status: 'waiting', expiresIn: data.expires_in });
    startCountdown(data.expires_in);
    startPolling(data.state);
  } catch (err) {
    if (!mountedRef.current) return;
    setState({
      qrUrl: null,
      status: 'expired',
      expiresIn: 0,
    });
    options.onLoginError(err instanceof Error ? err.message : '获取二维码失败');
  }
}, [apiClient, options, startCountdown, startPolling, stopAll]);

const refresh = useCallback(() => {
  void loadQrCode();
}, [loadQrCode]);

// Auto-load on mount
useEffect(() => {
  mountedRef.current = true;
  void loadQrCode();
  return () => {
    mountedRef.current = false;
```

```
      stopAll();
    };
    // Only run on mount/unmount
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

  return { ...state, refresh };
}



// ===== File: src/features/auth/components/WechatModal.tsx =====
import { Modal, Pressable, StyleSheet, Text, View } from 'react-native';

import { isWechatLoginSupported } from '../services/wechatNative';
import { LoginColors, LoginSizes, LoginWeights } from './LoginConstants';

type WechatModalProps = {
  visible: boolean;
  isSubmitting: boolean;
  onClose: () => void;
  onWechatLogin: () => void;
};

export function WechatModal({ visible, isSubmitting, onClose, onWechatLogin }: WechatModalPro
  const wechatSupported = isWechatLoginSupported();

  return (
    <Modal
      visible={visible}
      transparent
      animationType="fade"
      onRequestClose={onClose}
      statusBarTranslucent>
      <View style={styles.backdrop}>
        <View style={styles.dialog}>
          <Pressable style={styles.closeBtn} onPress={onClose} disabled={isSubmitting}>
            <Text style={styles.closeBtnText}>×</Text>
          </Pressable>

          <Text style={styles.title}>微信扫码登录</Text>

          <View style={styles.qrBox}>
            <View style={styles.qrPlaceholder}>
              <Text style={styles.qrPlaceholderIcon}></Text>
              <Text style={styles.qrPlaceholderText}>
                {wechatSupported
                  ? '点击下方按钮\n通过微信授权登录'
                  : '微信登录仅支持\niOS / Android 真机'}
              </Text>
            </View>
          </View>

          {wechatSupported ? (
            <Pressable
              style={[styles.wechatLoginBtn, isSubmitting && styles.disabled]}
              onPress={onWechatLogin}
              disabled={isSubmitting}>
              <Text style={styles.wechatLoginBtnText}>微信登录</Text>
            </Pressable>
          ) : null}

          <Text style={styles.meta}>
            {wechatSupported
```

```
              ? '请使用微信授权登录'
              : '请使用 development build 或正式包'}
          </Text>
        </View>
      </View>
    </Modal>
  );
}

const styles = StyleSheet.create({
  backdrop: {
    flex: 1,
    backgroundColor: LoginColors.modalBackdrop,
    alignItems: 'center',
    justifyContent: 'center',
    padding: 24,
  },
  dialog: {
    width: '100%',
    maxWidth: 360,
    paddingTop: 32,
    paddingBottom: 28,
    paddingHorizontal: 28,
    borderRadius: LoginSizes.modalBorderRadius,
    backgroundColor: LoginColors.modalDialogBg,
    alignItems: 'center',
    shadowColor: LoginColors.shadowColor,
    shadowOffset: { width: 0, height: 24 },
    shadowOpacity: 0.18,
    shadowRadius: 48,
    elevation: 12,
  },
  closeBtn: {
    position: 'absolute',
    top: 12,
    right: 12,
    width: LoginSizes.modalCloseSize,
    height: LoginSizes.modalCloseSize,
    borderRadius: LoginSizes.modalCloseSize / 2,
    backgroundColor: 'rgba(255,255,255,0.92)',
    alignItems: 'center',
    justifyContent: 'center',
  },
  closeBtnText: {
    fontSize: 28,
    lineHeight: 30,
    color: LoginColors.modalCloseBtn,
  },
  title: {
    marginBottom: 16,
    fontSize: 28,
    fontWeight: LoginWeights.extraBlack,
    color: LoginColors.wechatGreenModal,
    textAlign: 'center',
  },
  qrBox: {
    width: 260,
    aspectRatio: 1,
    padding: 14,
    borderWidth: 14,
    borderColor: 'rgba(255,255,255,0.96)',
    borderRadius: 24,
    backgroundColor: '#f4f6fa',
    overflow: 'hidden',
```

```
  },
  qrPlaceholder: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  qrPlaceholderIcon: {
    fontSize: 48,
    marginBottom: 12,
  },
  qrPlaceholderText: {
    fontSize: 15,
    fontWeight: LoginWeights.extraBold,
    color: '#888',
    textAlign: 'center',
    lineHeight: 22,
  },
  wechatLoginBtn: {
    marginTop: 16,
    width: 260,
    height: 48,
    borderRadius: 14,
    backgroundColor: LoginColors.wechatGreen,
    alignItems: 'center',
    justifyContent: 'center',
  },
  wechatLoginBtnText: {
    fontSize: 18,
    fontWeight: LoginWeights.black,
    color: LoginColors.white,
  },
  disabled: {
    opacity: 0.65,
  },
  meta: {
    marginTop: 14,
    fontSize: 14,
    fontWeight: LoginWeights.extraBold,
    color: '#666',
    textAlign: 'center',
  },
});
```

```
// ===== File: src/features/courses/components/CourseHomeScreen.tsx =====
import { Image } from 'expo-image';
import { Pressable, ScrollView, StyleSheet, Text, View, useWindowDimensions } from 'react-nat
import { useSafeAreaInsets } from 'react-native-safe-area-context';
import * as ScreenOrientation from 'expo-screen-orientation';
import { StatusBar } from 'expo-status-bar';
import { useCallback, useEffect } from 'react';
import { useFocusEffect } from 'expo-router';

import { useAuth } from '@/features/auth';
import { MAP_WIDTH } from '@/shared/courseHomeMap';

import { courseHomeImages } from '../assets/courseHomeAssets';
import { CourseEnterLoadingOverlay, CourseHomeLoadingState } from './CourseEnterLoadingOverla
import { CourseMapBackground } from './CourseMapBackground';
import { CourseNode } from './CourseNode';
import { CourseToolbar } from './CourseToolbar';
import { useCourseHome } from '../hooks/useCourseHome';
import { useLogoutTimer } from '../hooks/useLogoutTimer';
```

```
import {
  clampByViewport,
  computeContinueWidth,
  computeFoxWidth,
  computeTipWidth,
  isLandscapeTablet,
} from '../layout/courseHomeLayout';

export function CourseHomeScreen() {
  const { width, height } = useWindowDimensions();
  const insets = useSafeAreaInsets();
  const { auth, apiClient, logout } = useAuth();
  const landscapeTablet = isLandscapeTablet(width, height);

  const {
    scrollRef,
    isLoadingLessons,
    lessonLoadFailed,
    isEnteringCourse,
    isLoggingOut,
    totalCourses,
    mapHeight,
    mapPixelHeight,
    mapPixelWidth,
    completedSet,
    courseNodes,
    currentCourse,
    userName,
    handleCourseClick,
    handleContinue,
    handleLogout,
    refreshCourseHome,
  } = useCourseHome({
    apiClient,
    auth,
    logout,
    viewportWidth: width,
    viewportHeight: height,
  });

  const onForceLogout = useCallback(() => {
    void handleLogout();
  }, [handleLogout]);

  useLogoutTimer(apiClient, onForceLogout);

  useFocusEffect(
    useCallback(() => {
      void refreshCourseHome();
    }, [refreshCourseHome]),
  );

  useEffect(() => {
    void ScreenOrientation.unlockAsync();
    return () => {
      void ScreenOrientation.lockAsync(ScreenOrientation.OrientationLock.PORTRAIT_UP);
    };
  }, []);

  const countLabel = isLoadingLessons
    ? '正在加载课程...'
    : lessonLoadFailed
      ? '课程加载失败，请稍后重试'
      : `共 ${totalCourses} 节课程`;
```

```
const continueWidth = computeContinueWidth(width);
const tipWidth = computeTipWidth(width);
const tipFontSize = clampByViewport({ min: 12, vw: 0.01, max: 20 }, width);
const foxWidth = computeFoxWidth(width);

if (isLoadingLessons && totalCourses === 0 && !lessonLoadFailed) {
  return (
    <View style={styles.root}>
      <StatusBar style="light" />
      <CourseHomeLoadingState />
    </View>
  );
}

return (
  <View style={styles.root}>
    <StatusBar style="light" />

    <ScrollView
      ref={scrollRef}
      style={styles.scroll}
      contentContainerStyle={{
        minHeight: height,
        paddingBottom: insets.bottom,
      }}
      showsVerticalScrollIndicator={false}
      bounces={false}
    >
      <View
        style={{
          width: mapPixelWidth,
          height: mapPixelHeight,
          position: 'relative',
        }}
      >
        <CourseMapBackground
          width={mapPixelWidth}
          height={mapPixelHeight}
        />

        <View
          style={[
            styles.tip,
            {
              left: mapPixelWidth * 0.318,
              top: mapPixelHeight * 0.132,
              width: tipWidth,
            },
          ]}
          pointerEvents="none"
        >
          <Image source={courseHomeImages.tipBubble} style={styles.tipImage} contentFit="co
          <View style={styles.tipTextWrap}>
            <Text style={[styles.tipText, { fontSize: tipFontSize }]}>点击课程可重复学习哦！<
          </View>
        </View>

        {courseNodes.map((course) => {
          const completed = completedSet.has(course.number);
          const current = course.number === currentCourse?.number;
          const disabled = (!completed && !current) || isEnteringCourse;

          return (
```

```
        <CourseNode
          key={course.number}
          course={course}
          mapHeight={mapHeight}
          mapPixelHeight={mapPixelHeight}
          mapWidth={mapPixelWidth}
          viewportWidth={width}
          completed={completed}
          current={current}
          disabled={disabled}
          onPress={() => {
            if (current) {
              handleContinue();
            } else {
              handleCourseClick(course);
            }
          }}
        />
      );
    })}

    {currentCourse ? (
      <Image
        source={courseHomeImages.fox}
        style={[
          styles.fox,
          {
            width: foxWidth,
            height: foxWidth * 1.1,
            left:
              (currentCourse.x / MAP_WIDTH) * mapPixelWidth - foxWidth / 2,
            top:
              ((currentCourse.y - 185) / mapHeight) * mapPixelHeight - foxWidth / 2,
          },
        ]}
        contentFit="contain"
      />
    ) : null}
  </View>
</ScrollView>

<CourseToolbar
  viewportWidth={width}
  landscapeTablet={landscapeTablet}
  countLabel={countLabel}
  userName={userName}
  currentCourseNumber={currentCourse?.number || 1}
  isLoggingOut={isLoggingOut}
  onLogout={() => {
    void handleLogout();
  }}
/>

<Pressable
  accessibilityRole="button"
  accessibilityLabel="继续学习"
  disabled={!currentCourse || isEnteringCourse}
  onPress={handleContinue}
  style={({ pressed }) => [
    styles.continue,
    {
      width: continueWidth,
      bottom: insets.bottom + 36,
      opacity: !currentCourse || isEnteringCourse ? 0.72 : 1,
```

```
          },
          pressed && currentCourse && !isEnteringCourse && styles.continuePressed,
        ]}
      >
        <Image
          source={courseHomeImages.continueButton}
          style={styles.continueImage}
          contentFit="contain"
        />
      </Pressable>

      <CourseEnterLoadingOverlay visible={isEnteringCourse} viewportWidth={width} />
    </View>
  );
}

const styles = StyleSheet.create({
  root: {
    flex: 1,
    backgroundColor: '#25b8dd',
  },
  scroll: {
    flex: 1,
  },
  tip: {
    position: 'absolute',
    zIndex: 3,
  },
  tipImage: {
    width: '100%',
    aspectRatio: 2.2,
  },
  tipTextWrap: {
    position: 'absolute',
    left: 0,
    right: 0,
    top: 0,
    bottom: 0,
    paddingLeft: '8%',
    alignItems: 'center',
    justifyContent: 'center',
  },
  tipText: {
    color: '#426ae6',
    fontWeight: '900',
  },
  fox: {
    position: 'absolute',
    zIndex: 5,
    shadowColor: '#164011',
    shadowOpacity: 0.22,
    shadowRadius: 10,
    shadowOffset: { width: 0, height: 16 },
    elevation: 4,
  },
  continue: {
    position: 'absolute',
    alignSelf: 'center',
    zIndex: 20,
    backgroundColor: 'transparent',
  },
  continuePressed: {
    transform: [{ translateY: -2 }],
  },
```

```
  continueImage: {
    width: '100%',
    aspectRatio: 3.2,
    shadowColor: '#6f4703',
    shadowOpacity: 0.18,
    shadowRadius: 18,
    shadowOffset: { width: 0, height: 10 },
  },
});
```

```
// ===== File: src/features/courses/components/CourseMapBackground.tsx =====
import { Image } from 'expo-image';
import { StyleSheet, View } from 'react-native';

import { courseHomeImages } from '../assets/courseHomeAssets';
import {
  computeBackgroundTileCount,
  computeBackgroundTileHeight,
} from '../layout/courseHomeLayout';

type CourseMapBackgroundProps = {
  width: number;
  height: number;
};

export function CourseMapBackground({ width, height }: CourseMapBackgroundProps) {
  const tileHeight = computeBackgroundTileHeight(width);
  const tileCount = computeBackgroundTileCount(width, height);

  return (
    <View style={[styles.container, { width, height }]} pointerEvents="none">
      {Array.from({ length: tileCount }, (_, index) => (
        <Image
          key={index}
          source={courseHomeImages.background}
          style={{
            position: 'absolute',
            top: index * tileHeight,
            width,
            height: tileHeight,
          }}
          contentFit="fill"
        />
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    position: 'absolute',
    left: 0,
    top: 0,
    backgroundColor: '#25b8dd',
    zIndex: 0,
  },
});
```

```
// ===== File: src/features/courses/components/CourseNode.tsx =====
import { Image } from 'expo-image';
```

```
import { Pressable, StyleSheet, Text } from 'react-native';

import { getCourseButtonImageSource, courseHomeImages } from '../assets/courseHomeAssets';
import { clampByViewport } from '../layout/courseHomeLayout';
import { MAP_WIDTH, type CourseMapNode } from '@/shared/courseHomeMap';

type CourseNodeProps = {
  course: CourseMapNode;
  mapHeight: number;
  mapPixelHeight: number;
  mapWidth: number;
  viewportWidth: number;
  completed: boolean;
  current: boolean;
  disabled: boolean;
  onPress: () => void;
};

export function CourseNode({
  course,
  mapHeight,
  mapPixelHeight,
  mapWidth,
  viewportWidth,
  completed,
  current,
  disabled,
  onPress,
}: CourseNodeProps) {
  const nodeWidth = Math.max(62, mapWidth * 0.0662);
  const numberSize = clampByViewport({ min: 20, vw: 0.031, max: 62 }, viewportWidth);

  const centerX = (course.x / MAP_WIDTH) * mapWidth;
  const centerY = (course.y / mapHeight) * mapPixelHeight;

  return (
    <Pressable
      accessibilityRole="button"
      accessibilityLabel={`${course.title}${completed ? '，已完成，可复习' : current ? '，当前课
      disabled={disabled}
      onPress={onPress}
      style={({ pressed }) => [
        styles.node,
        {
          width: nodeWidth,
          left: centerX - nodeWidth / 2,
          top: centerY - nodeWidth / 2,
          zIndex: current ? 4 : 2,
        },
        completed && styles.completed,
        !completed && !current && styles.locked,
        !disabled && pressed && styles.pressed,
      ]}
    >
      <Image
        source={getCourseButtonImageSource(completed)}
        style={styles.nodeImage}
        contentFit="contain"
      />
      <Text
        style={[
          styles.number,
          { fontSize: numberSize },
          completed ? styles.numberCompleted : styles.numberLocked,
```

```
        ]}
      >
        {course.number}
      </Text>
      {completed ? (
        <Image source={courseHomeImages.stars} style={styles.stars} contentFit="contain" />
      ) : null}
    </Pressable>
  );
}

const styles = StyleSheet.create({
  node: {
    position: 'absolute',
    backgroundColor: 'transparent',
    padding: 0,
  },
  completed: {},
  locked: {
    opacity: 0.98,
  },
  pressed: {
    transform: [{ translateY: -4 }, { scale: 1.06 }],
  },
  nodeImage: {
    width: '100%',
    aspectRatio: 1,
  },
  number: {
    position: 'absolute',
    left: 0,
    right: 0,
    top: '21%',
    color: '#ffffff',
    fontWeight: '900',
    lineHeight: undefined,
    textAlign: 'center',
    textShadowColor: 'rgba(0, 0, 0, 0.2)',
    textShadowOffset: { width: 0, height: 4 },
    textShadowRadius: 8,
  },
  numberCompleted: {
    textShadowColor: 'rgba(54, 125, 29, 0.38)',
  },
  numberLocked: {
    textShadowColor: 'rgba(84, 92, 104, 0.42)',
  },
  stars: {
    position: 'absolute',
    left: '14%',
    bottom: '18%',
    width: '72%',
    aspectRatio: 3,
  },
});



// ===== File: src/features/lesson/webViewMessages.ts =====
import type { FtAuthRecord } from '@/lib/auth/types';

export type WebViewBridgeMessage = {
  version?: number;
  messageType: number;
```

```
  payload?: string | null;
};

export type WebViewAuthUpdate =
  | { kind: 'login'; auth: FtAuthRecord }
  | { kind: 'logout' }
  | { kind: 'update_user'; patch: Partial<FtAuthRecord> };

export function parseWebViewBridgeMessage(raw: string): WebViewBridgeMessage | null {
  try {
    const parsed = JSON.parse(raw) as WebViewBridgeMessage;
    if (typeof parsed.messageType !== 'number') {
      return null;
    }
    return parsed;
  } catch {
    return null;
  }
}

function parsePayloadObject(payload: string | null | undefined): Record<string, unknown> | nu
  if (!payload) {
    return null;
  }
  try {
    const parsed = JSON.parse(payload) as Record<string, unknown>;
    return parsed && typeof parsed === 'object' ? parsed : null;
  } catch {
    return null;
  }
}

export function resolveWebViewAuthUpdate(
  message: WebViewBridgeMessage,
  currentAuth: FtAuthRecord | null,
): WebViewAuthUpdate | null {
  switch (message.messageType) {
    case 1: {
      const payload = parsePayloadObject(message.payload);
      if (!payload?.token || typeof payload.token !== 'string') {
        return null;
      }
      const username =
        (typeof payload.username === 'string' && payload.username) ||
        (typeof payload.name === 'string' && payload.name) ||
        '';
      return {
        kind: 'login',
        auth: {
          userId:
            typeof payload.userId === 'string'
              ? payload.userId
              : payload.userId != null
                ? String(payload.userId)
                : currentAuth?.userId || '',
          token: payload.token,
          hasUsername: Boolean(username),
          username,
          name: username,
          phone: typeof payload.phone === 'string' ? payload.phone : currentAuth?.phone || ''
          logo: typeof payload.logo === 'string' ? payload.logo : currentAuth?.logo || '',
          persistent: currentAuth?.persistent ?? true,
          expiresAt: currentAuth?.expiresAt,
          authType: currentAuth?.authType,
```

```
        },
      };
    }
    case 3:
      return { kind: 'logout' };
    case 5: {
      const payload = parsePayloadObject(message.payload);
      const username = typeof payload?.username === 'string' ? payload.username : '';
      return {
        kind: 'update_user',
        patch: {
          username,
          name: username,
          hasUsername: Boolean(username.trim()),
        },
      };
    }
    default:
      return null;
  }
}



// ===== File: src/features/lesson/buildLessonWebUrl.ts =====
export type LessonRouteParams = {
  lesson_id?: string;
  section_id?: string;
  course_number?: string;
  total_courses?: string;
  from?: string;
  skip_opening?: string;
  autostart?: string;
  web_destination?: string;
  web_base_url?: string;
};

export function normalizeRouteParam(value: string | string[] | undefined): string | undefined
  if (Array.isArray(value)) {
    return value[0];
  }
  return value;
}

export function buildLessonWebDestination(params: LessonRouteParams): string {
  const explicitDestination = normalizeRouteParam(params.web_destination)?.trim();
  if (explicitDestination) {
    return explicitDestination.startsWith('/') ? explicitDestination : `/${explicitDestinatio
  }

  const search = new URLSearchParams();
  const lessonId = normalizeRouteParam(params.lesson_id)?.trim();
  if (lessonId) {
    search.set('lesson_id', lessonId);
  }

  const sectionId = normalizeRouteParam(params.section_id)?.trim();
  if (sectionId) {
    search.set('section_id', sectionId);
  }

  const from = normalizeRouteParam(params.from)?.trim();
  if (from) {
    search.set('from', from);
```

```
  }

  const courseNumber = normalizeRouteParam(params.course_number)?.trim();
  if (courseNumber) {
    search.set('course_number', courseNumber);
  }

  const totalCourses = normalizeRouteParam(params.total_courses)?.trim();
  if (totalCourses) {
    search.set('total_courses', totalCourses);
  }

  const autostart = normalizeRouteParam(params.autostart)?.trim();
  if (autostart) {
    search.set('autostart', autostart);
  }

  const skipOpening = normalizeRouteParam(params.skip_opening)?.trim();
  if (skipOpening) {
    search.set('skip_opening', skipOpening);
  }

  const query = search.toString();
  return query ? `/app/lesson?${query}` : '/app/lesson';
}

export function buildLessonWebUrl(webBaseUrl: string, destinationPath: string): string {
  const base = webBaseUrl.replace(/\/$/, '');
  if (/^https?:\/\//i.test(destinationPath)) {
    return destinationPath;
  }
  return `${base}${destinationPath.startsWith('/') ? destinationPath : `/${destinationPath}`}
}

export function isCourseHomeWebUrl(url: string): boolean {
  try {
    const pathname = new URL(url).pathname.replace(/\/$/, '');
    return pathname === '/app/courses';
  } catch {
    return url.includes('/app/courses');
  }
}



// ===== File: src/features/lesson/hooks/useNativeLessonController.ts =====
import { useCallback, useEffect, useMemo, useReducer } from 'react';

import {
  buildNativeLessonControllerItems,
  createNativeLessonControllerState,
  getNativeLessonControllerView,
  reduceNativeLessonController,
} from '../nativeLessonController';
import { getNativeLessonMediaPreloadUris } from '../nativeLessonMedia';
import type { NativeLessonDefinition } from '../nativeLessonTypes';

export function useNativeLessonController(lesson: NativeLessonDefinition) {
  const [state, dispatch] = useReducer(
    (current: ReturnType<typeof createNativeLessonControllerState>, action: Parameters<typeof
      reduceNativeLessonController(lesson, current, action),
    lesson,
    createNativeLessonControllerState,
  );
```

```
  useEffect(() => {
    dispatch({ type: 'reset' });
  }, [lesson]);

  const view = useMemo(
    () => getNativeLessonControllerView(lesson, state),
    [lesson, state],
  );
  const preloadUris = useMemo(
    () =>
      getNativeLessonMediaPreloadUris(
        buildNativeLessonControllerItems(lesson),
        state.snapshot.currentIndex,
      ),
    [lesson, state.snapshot.currentIndex],
  );

  return {
    state,
    view,
    preloadUris,
    next: useCallback(() => dispatch({ type: 'next' }), []),
    submitChoice: useCallback(
      (optionId: string) => dispatch({ type: 'submit_choice', optionId }),
      [],
    ),
    submitText: useCallback(
      (text: string) => dispatch({ type: 'submit_text', text }),
      [],
    ),
    pause: useCallback(() => dispatch({ type: 'pause' }), []),
    resume: useCallback(() => dispatch({ type: 'resume' }), []),
    reset: useCallback(() => dispatch({ type: 'reset' }), []),
  };
}



// ===== File: src/features/lesson/hooks/useNativeLessonRecording.ts =====
import { useCallback, useEffect, useRef, useReducer } from 'react';
import { AppState } from 'react-native';

import {
  createRecordingControllerState,
  reduceRecordingController,
} from '../recordingController';
import type { SimpleVadConfig } from '../simpleVad';
import { loadNativeExpoAv } from '../nativeExpoAv';

type NativeRecordingStatus = {
  isRecording?: boolean;
  metering?: number;
  durationMillis?: number;
};

type UseNativeLessonRecordingOptions = {
  vadConfig?: Partial<SimpleVadConfig>;
};

export function useNativeLessonRecording(options?: UseNativeLessonRecordingOptions) {
  const recordingRef = useRef<{
    stopAndUnloadAsync: () => Promise<{ durationMillis?: number }>;
    getURI: () => string | null;
```

```
} | null>(null);
const [state, dispatch] = useReducer(
  reduceRecordingController,
  options?.vadConfig,
  createRecordingControllerState,
);

const stop = useCallback(async () => {
  const recording = recordingRef.current;
  if (!recording) {
    return;
  }
  recordingRef.current = null;
  try {
    const status = await recording.stopAndUnloadAsync();
    dispatch({
      type: 'stop',
      uri: recording.getURI(),
      durationMs: status.durationMillis ?? state.durationMs,
    });
  } catch (error) {
    dispatch({
      type: 'error',
      message: error instanceof Error ? error.message : '录音停止失败。',
    });
  }
}, [state.durationMs]);

const start = useCallback(async () => {
  try {
    const { Audio } = await loadNativeExpoAv();
    const permission = await Audio.requestPermissionsAsync();
    if (!permission.granted) {
      dispatch({ type: 'permission_denied' });
      return;
    }
    dispatch({ type: 'permission_granted' });
    await Audio.setAudioModeAsync({
      allowsRecordingIOS: true,
      playsInSilentModeIOS: true,
    });
    const { recording } = await Audio.Recording.createAsync(
      Audio.RecordingOptionsPresets.HIGH_QUALITY,
      (status: NativeRecordingStatus) => {
        if (status.isRecording) {
          dispatch({
            type: 'metering',
            metering: status.metering,
            elapsedMs: status.durationMillis ?? 0,
          });
        }
      },
      200,
    );
    recordingRef.current = recording;
    dispatch({ type: 'start' });
  } catch (error) {
    dispatch({
      type: 'error',
      message: error instanceof Error ? error.message : '录音启动失败。',
    });
  }
}, []);
```

```
  const cancel = useCallback(async () => {
    const recording = recordingRef.current;
    recordingRef.current = null;
    try {
      await recording?.stopAndUnloadAsync();
    } catch {
      // The recording may already be unloaded by auto-stop; cancellation should still clear
    }
    dispatch({ type: 'cancel' });
  }, []);

  const submit = useCallback(() => {
    dispatch({ type: 'submit' });
  }, []);

  useEffect(() => {
    if (state.shouldStopRecording) {
      stop();
    }
  }, [state.shouldStopRecording, stop]);

  useEffect(
    () => () => {
      const recording = recordingRef.current;
      recordingRef.current = null;
      recording?.stopAndUnloadAsync().catch(() => {});
    },
    [],
  );

  useEffect(() => {
    const subscription = AppState.addEventListener('change', (nextState) => {
      if (nextState !== 'active' && recordingRef.current) {
        void stop();
      }
    });

    return () => subscription.remove();
  }, [stop]);

  return {
    state,
    start,
    stop,
    cancel,
    submit,
  };
}



// ===== File: src/features/lesson/hooks/useNativeLessonRealtimeSession.ts =====
import { useCallback, useEffect, useMemo, useRef, useState } from 'react';

import { getDeviceID } from '@/lib/device/deviceId';

import { unpackRealtimeWireFrame } from '../nativeRealtimeAudio';
import {
  INITIAL_REALTIME_LESSON_PROJECTION_STATE,
  applyRealtimeLessonEvent,
  getRealtimeControllerView,
  type RealtimeLessonProjectionState,
} from '../nativeLessonRealtimeProjection';
import {
```

```
  buildAssistantPromptSpokenCommand,
  buildInitSessionCommand,
  buildMediaFinishedCommand,
  buildRealtimeLessonWsUrl,
  buildRequestDebugNextStepCommand,
  buildStartLessonCommand,
  buildSubmitChoiceCommand,
  buildSubmitUserTextCommand,
  decodeRealtimeLessonMessage,
  normalizeRealtimeLessonEvent,
} from '../nativeLessonSessionProtocol';
import type {
  NativeLessonControllerView,
  NativeLessonPhase,
} from '../nativeLessonController';
import { shouldUseServerOnlyTts } from '../nativeLessonTtsPolicy';
import { useNativeRealtimeAudioPlayback } from './useNativeRealtimeAudioPlayback';

type NativeLessonRealtimeSessionOptions = {
  enabled: boolean;
  apiBaseUrl: string;
  token: string;
  lessonId: string;
  sectionId?: string;
  title: string;
  backgroundImageUrl?: string;
};

type RealtimeSessionStatus = 'idle' | 'connecting' | 'connected' | 'disconnected' | 'error';

const MAX_RECONNECT_ATTEMPTS = 5;
const INITIAL_RECONNECT_DELAY_MS = 800;
const MAX_RECONNECT_DELAY_MS = 8000;

function sendJson(socket: WebSocket | null, payload: unknown): boolean {
  if (!socket || socket.readyState !== WebSocket.OPEN) {
    return false;
  }
  socket.send(JSON.stringify(payload));
  return true;
}

export function useNativeLessonRealtimeSession(options: NativeLessonRealtimeSessionOptions) {
  const socketRef = useRef<WebSocket | null>(null);
  const closedByUserRef = useRef(false);
  const reconnectTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const ttsFallbackTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const localTtsFallbackTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);
  const sessionIdRef = useRef<string | null>(null);
  const forceFreshSessionRef = useRef(false);
  const currentStepIdRef = useRef<number | null>(null);
  const currentStepPhaseRef = useRef<NativeLessonPhase | null>(null);
  const currentAssistantPromptRef = useRef('');
  const ttsReceivedAudioRef = useRef(false);
  const reconnectAttemptRef = useRef(0);
  const spokenStepIdsRef = useRef(new Set<number>());
  const [status, setStatus] = useState<RealtimeSessionStatus>('idle');
  const [projection, setProjection] = useState<RealtimeLessonProjectionState>(
    INITIAL_REALTIME_LESSON_PROJECTION_STATE,
  );
  const [errorText, setErrorText] = useState('');
  const markAssistantPromptSpoken = useCallback(() => {
    const stepId = currentStepIdRef.current;
    if (typeof stepId !== 'number' || spokenStepIdsRef.current.has(stepId)) {
```

```
    return;
  }
  spokenStepIdsRef.current.add(stepId);
  sendJson(socketRef.current, buildAssistantPromptSpokenCommand(stepId));
}, []);
const audioPlayback = useNativeRealtimeAudioPlayback(() => {
  markAssistantPromptSpoken();
});
const {
  pushPcmChunk,
  playBufferedPcm,
  playRemoteUrl,
  speakTextFallback,
  resetBuffer,
  status: audioStatus,
  errorText: audioErrorText,
} = audioPlayback;

const clearSessionState = useCallback(() => {
  sessionIdRef.current = null;
  currentStepIdRef.current = null;
  currentStepPhaseRef.current = null;
  currentAssistantPromptRef.current = '';
  ttsReceivedAudioRef.current = false;
  reconnectAttemptRef.current = 0;
  spokenStepIdsRef.current.clear();
  setProjection(INITIAL_REALTIME_LESSON_PROJECTION_STATE);
  setErrorText('');
  setStatus('idle');
  resetBuffer();
}, [resetBuffer]);

const realtimeView = useMemo<NativeLessonControllerView | null>(
  () =>
    getRealtimeControllerView(projection, {
      title: options.title,
      backgroundImageUrl: options.backgroundImageUrl,
    }),
  [options.backgroundImageUrl, options.title, projection],
);

const closeSocket = useCallback(() => {
  closedByUserRef.current = true;
  if (reconnectTimerRef.current) {
    clearTimeout(reconnectTimerRef.current);
    reconnectTimerRef.current = null;
  }
  if (ttsFallbackTimerRef.current) {
    clearTimeout(ttsFallbackTimerRef.current);
    ttsFallbackTimerRef.current = null;
  }
  if (localTtsFallbackTimerRef.current) {
    clearTimeout(localTtsFallbackTimerRef.current);
    localTtsFallbackTimerRef.current = null;
  }
  socketRef.current?.close();
  socketRef.current = null;
}, []);

const connect = useCallback(async () => {
  if (!options.enabled || !options.token) {
    return;
  }
  closedByUserRef.current = false;
```

```
setStatus('connecting');
setErrorText('');

try {
  const deviceId = await getDeviceID();
  const socket = new WebSocket(
    buildRealtimeLessonWsUrl(options.apiBaseUrl, deviceId, options.token),
  );
  socket.binaryType = 'arraybuffer';
  socketRef.current = socket;

  socket.onopen = () => {
    reconnectAttemptRef.current = 0;
    setStatus('connected');
    sendJson(
      socket,
      buildInitSessionCommand({
        lessonId: options.lessonId,
        sectionId: options.sectionId,
        token: options.token,
        resumeSessionId: forceFreshSessionRef.current ? null : sessionIdRef.current,
        allowOwnerResume: forceFreshSessionRef.current ? false : true,
      }),
    );
    forceFreshSessionRef.current = false;
    sendJson(socket, buildStartLessonCommand());
  };

  socket.onmessage = (message) => {
    void (async () => {
      const wireFrame = unpackRealtimeWireFrame(message.data);
      if (wireFrame?.kind === 'audio') {
        ttsReceivedAudioRef.current = true;
        if (localTtsFallbackTimerRef.current) {
          clearTimeout(localTtsFallbackTimerRef.current);
          localTtsFallbackTimerRef.current = null;
        }
        pushPcmChunk(wireFrame.payload);
        return;
      }
      const decoded =
        wireFrame?.kind === 'json'
          ? wireFrame.payload
          : await decodeRealtimeLessonMessage(message.data);
      const event = normalizeRealtimeLessonEvent(decoded);
      if (!event) {
        return;
      }
      if (event.event === 'session_ready' && event.sessionId) {
        sessionIdRef.current = event.sessionId;
      }
      if (event.event === 'lesson_state_snapshot' && event.currentStepId) {
        currentStepIdRef.current = event.currentStepId;
        currentStepPhaseRef.current =
          event.phase === 'story' ||
          event.phase === 'teaching' ||
          event.phase === 'challenge' ||
          event.phase === 'free_chat'
            ? event.phase
            : currentStepPhaseRef.current;
      }
      if (event.event === 'step_started') {
        currentStepIdRef.current = event.step.stepId;
        currentStepPhaseRef.current =
```

```
        event.step.phase === 'story' ||
        event.step.phase === 'teaching' ||
        event.step.phase === 'challenge' ||
        event.step.phase === 'free_chat'
          ? event.step.phase
          : null;
      currentAssistantPromptRef.current = event.step.assistantPrompt;
      spokenStepIdsRef.current.delete(event.step.stepId);
      if (event.step.voiceUrl) {
        void playRemoteUrl(event.step.voiceUrl);
      }
    }
    if (event.event === 'assistant_message' || event.event === 'assistant_speech_start'
      if (event.text.trim()) {
        currentAssistantPromptRef.current = event.text.trim();
      }
    }
    if (event.event === 'tts_start') {
      if (ttsFallbackTimerRef.current) {
        clearTimeout(ttsFallbackTimerRef.current);
      }
      if (localTtsFallbackTimerRef.current) {
        clearTimeout(localTtsFallbackTimerRef.current);
      }
      ttsReceivedAudioRef.current = false;
      ttsFallbackTimerRef.current = setTimeout(() => {
        ttsFallbackTimerRef.current = null;
        markAssistantPromptSpoken();
      }, 20000);
      if (!shouldUseServerOnlyTts(currentStepPhaseRef.current)) {
        localTtsFallbackTimerRef.current = setTimeout(() => {
          localTtsFallbackTimerRef.current = null;
          if (!ttsReceivedAudioRef.current) {
            void speakTextFallback(currentAssistantPromptRef.current);
          }
        }, 4500);
      }
      resetBuffer();
      return;
    }
    if (event.event === 'tts_end') {
      if (ttsFallbackTimerRef.current) {
        clearTimeout(ttsFallbackTimerRef.current);
        ttsFallbackTimerRef.current = null;
      }
      if (localTtsFallbackTimerRef.current) {
        clearTimeout(localTtsFallbackTimerRef.current);
        localTtsFallbackTimerRef.current = null;
      }
      if (ttsReceivedAudioRef.current) {
        await playBufferedPcm();
      } else if (shouldUseServerOnlyTts(currentStepPhaseRef.current)) {
        setErrorText('Server TTS 未返回音频，请重试。');
        markAssistantPromptSpoken();
      } else {
        await speakTextFallback(currentAssistantPromptRef.current);
      }
      return;
    }
    setProjection((current) => applyRealtimeLessonEvent(current, event));
  })();
};

socket.onerror = () => {
```

```
      setStatus('error');
      setErrorText('Realtime session 连接失败。');
    };

    socket.onclose = () => {
      setStatus((current) => (current === 'error' ? 'error' : 'disconnected'));
      socketRef.current = null;
      if (!closedByUserRef.current && options.enabled) {
        if (reconnectAttemptRef.current >= MAX_RECONNECT_ATTEMPTS) {
          setStatus('error');
          setErrorText('Realtime session 多次重连失败，请重试或切换到 WebView。');
          return;
        }
        const delay = Math.min(
          INITIAL_RECONNECT_DELAY_MS * 2 ** reconnectAttemptRef.current,
          MAX_RECONNECT_DELAY_MS,
        );
        reconnectAttemptRef.current += 1;
        reconnectTimerRef.current = setTimeout(() => {
          void connect();
        }, delay);
      }
    };
  } catch (error) {
    setStatus('error');
    setErrorText(error instanceof Error ? error.message : 'Realtime session 启动失败。');
  }
}, [
  options.apiBaseUrl,
  options.enabled,
  options.lessonId,
  options.sectionId,
  options.token,
  markAssistantPromptSpoken,
  playBufferedPcm,
  playRemoteUrl,
  pushPcmChunk,
  resetBuffer,
  speakTextFallback,
]);

useEffect(() => {
  clearSessionState();
  if (options.enabled) {
    void connect();
  }
  return closeSocket;
}, [clearSessionState, closeSocket, connect, options.enabled]);

const sendText = useCallback((text: string, stepId?: number) => {
  return sendJson(socketRef.current, buildSubmitUserTextCommand(text, stepId));
}, []);

const sendChoice = useCallback((stepId: number, optionId: string) => {
  return sendJson(socketRef.current, buildSubmitChoiceCommand(stepId, optionId));
}, []);

const sendMediaFinished = useCallback((cueId?: string, summary?: string) => {
  return sendJson(socketRef.current, buildMediaFinishedCommand({ cueId, summary }));
}, []);

const sendAssistantPromptSpoken = useCallback((stepId: number) => {
  return sendJson(socketRef.current, buildAssistantPromptSpokenCommand(stepId));
}, []);
```

```
  const requestDebugNextStep = useCallback(() => {
    return sendJson(socketRef.current, buildRequestDebugNextStepCommand());
  }, []);

  const sendAudioChunk = useCallback((chunk: ArrayBuffer | Uint8Array) => {
    const socket = socketRef.current;
    if (!socket || socket.readyState !== WebSocket.OPEN) {
      return false;
    }
    socket.send(chunk);
    return true;
  }, []);

  const reset = useCallback(() => {
    closeSocket();
    clearSessionState();
    if (options.enabled) {
      forceFreshSessionRef.current = true;
      closedByUserRef.current = false;
      setStatus('connecting');
      setTimeout(() => {
        void connect();
      }, 0);
    }
  }, [clearSessionState, closeSocket, connect, options.enabled]);

  return {
    status,
    errorText: errorText || projection.errorText,
    audioStatus,
    audioErrorText,
    projection,
    realtimeView,
    isConnected: status === 'connected',
    connect,
    close: closeSocket,
    reset,
    sendText,
    sendChoice,
    sendMediaFinished,
    sendAssistantPromptSpoken,
    requestDebugNextStep,
    sendAudioChunk,
  };
}



// ===== File: src/lib/api/client.ts =====
import { getDeviceID } from '@/lib/device/deviceId';

const DEFAULT_REQUEST_TIMEOUT_MS = 15_000;

export class ApiRequestError extends Error {
  constructor(message: string) {
    super(message);
    this.name = 'ApiRequestError';
  }
}

export type ApiClient = {
  baseUrl: string;
  request: (path: string, init?: RequestInit) => Promise<Response>;
```

```
  get: (path: string, init?: RequestInit) => Promise<Response>;
  post: (path: string, body?: unknown, init?: RequestInit) => Promise<Response>;
};

export type CreateApiClientOptions = {
  baseUrl: string;
  getDeviceId?: () => Promise<string>;
  getAccessToken?: () => string | null | Promise<string | null>;
  onUnauthorized?: () => void;
  requestTimeoutMs?: number;
};

function appendDeviceId(url: URL, deviceId: string) {
  if (!url.searchParams.has('deviceID')) {
    url.searchParams.append('deviceID', deviceId);
  }
}

function buildNetworkErrorMessage(baseUrl: string, error: unknown): string {
  if (error instanceof Error && error.name === 'AbortError') {
    return `连接超时，请确认后端已启动且 EXPO_PUBLIC_API_HOST 配置正确（当前：${baseUrl}）`;
  }

  return `无法连接服务器，请确认后端已启动且 EXPO_PUBLIC_API_HOST 配置正确（当前：${baseUrl}）`;
}

export function createApiClient(options: CreateApiClientOptions): ApiClient {
  const getDeviceId = options.getDeviceId ?? getDeviceID;
  const requestTimeoutMs = options.requestTimeoutMs ?? DEFAULT_REQUEST_TIMEOUT_MS;

  async function request(path: string, init?: RequestInit): Promise<Response> {
    const url = new URL(path, options.baseUrl);
    appendDeviceId(url, await getDeviceId());

    const headers = new Headers(init?.headers ?? {});
    if (!headers.has('Accept')) {
      headers.set('Accept', 'application/json');
    }

    const accessToken = options.getAccessToken ? await options.getAccessToken() : null;
    if (accessToken && !headers.has('Authorization')) {
      headers.set('Authorization', `Bearer ${accessToken}`);
    }

    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), requestTimeoutMs);

    let response: Response;
    try {
      response = await fetch(url.toString(), {
        ...init,
        credentials: 'include',
        headers,
        signal: controller.signal,
      });
    } catch (error) {
      throw new ApiRequestError(buildNetworkErrorMessage(options.baseUrl, error));
    } finally {
      clearTimeout(timeoutId);
    }

    if (response.status === 401 && options.onUnauthorized) {
      options.onUnauthorized();
    }
```

```
    return response;
  }

  return {
    baseUrl: options.baseUrl,
    request,
    get: (path, init) => request(path, { ...init, method: 'GET' }),
    post: (path, body, init) => {
      const headers = new Headers(init?.headers ?? {});
      if (body !== undefined && !headers.has('Content-Type')) {
        headers.set('Content-Type', 'application/json');
      }

      return request(path, {
        ...init,
        method: 'POST',
        headers,
        body: body === undefined ? undefined : JSON.stringify(body),
      });
    },
  };
}

export function withAccessToken(apiClient: ApiClient, token: string): ApiClient {
  return createApiClient({
    baseUrl: apiClient.baseUrl,
    getAccessToken: () => token,
  });
}



// ===== File: src/lib/auth/session.ts =====
import type { FtAuthRecord } from './types';

export type LoginUserInfo = {
  openid?: string;
  username?: string;
  phone?: string;
  avatar?: string;
  auth_type?: string;
};

export type LoginSuccessResponse = {
  success?: boolean;
  token?: string;
  expires_in?: number;
  remember_me?: boolean;
  user_info?: LoginUserInfo;
};

export type CheckSessionResponse = {
  code?: number;
  token?: string;
  expires_in?: number;
  user_info?: LoginUserInfo;
};

const PERSISTENT_AUTH_MS = 7 * 24 * 60 * 60 * 1000;
const SESSION_AUTH_MS = 24 * 60 * 60 * 1000;

export function buildFtAuthFromLoginResponse(
  data: LoginSuccessResponse,
```

```
  rememberMe: boolean,
): FtAuthRecord {
  const user = data.user_info ?? {};
  const expiresInMs =
    Number(data.expires_in) > 0
      ? Number(data.expires_in) * 1000
      : rememberMe
        ? PERSISTENT_AUTH_MS
        : SESSION_AUTH_MS;

  const username = user.username || '';
  return {
    userId: user.openid || '',
    token: data.token || '',
    hasUsername: Boolean(username),
    username,
    name: username,
    phone: user.phone || '',
    logo: user.avatar || '',
    authType: user.auth_type || '',
    persistent: rememberMe,
    expiresAt: Date.now() + expiresInMs,
  };
}

export function buildFtAuthFromCheckResponse(
  data: CheckSessionResponse,
  previous?: FtAuthRecord | null,
): FtAuthRecord {
  const user = data.user_info ?? {};
  const rememberMe = previous?.persistent ?? true;
  const expiresInMs =
    Number(data.expires_in) > 0
      ? Number(data.expires_in) * 1000
      : rememberMe
        ? PERSISTENT_AUTH_MS
        : SESSION_AUTH_MS;
  const username = user.username || previous?.username || '';

  return {
    userId: user.openid || previous?.userId || '',
    token: data.token || previous?.token || '',
    hasUsername: Boolean(username),
    username,
    name: username,
    phone: user.phone || previous?.phone || '',
    logo: user.avatar || previous?.logo || '',
    authType: previous?.authType || '',
    persistent: rememberMe,
    expiresAt: Date.now() + expiresInMs,
  };
}

export function mergeAuthRecord(
  base: FtAuthRecord,
  patch: Partial<FtAuthRecord>,
): FtAuthRecord {
  return { ...base, ...patch };
}



// ===== File: src/lib/auth/storage.ts =====
import { Platform } from 'react-native';
```

```
import * as SecureStore from 'expo-secure-store';

import { defaultAsyncStorage, type KeyValueStorage } from '@/lib/storage/asyncStorage';

import type { FtAuthRecord } from './types';
import {
  clearFtAuthFromStores,
  getFtAuthFromStores,
  setFtAuthToStores,
  type TokenStore,
} from './storageCore';

const FT_AUTH_TOKEN_KEY = 'ft_auth_token';
const FT_AUTH_PROFILE_KEY = 'ft_auth_profile';

function createTokenStore(storage: KeyValueStorage = defaultAsyncStorage): TokenStore {
  if (Platform.OS === 'web') {
    return {
      get: () => storage.getItem(FT_AUTH_TOKEN_KEY),
      set: async (value) => {
        if (value) {
          await storage.setItem(FT_AUTH_TOKEN_KEY, value);
          return;
        }
        await storage.removeItem(FT_AUTH_TOKEN_KEY);
      },
    };
  }

  return {
    get: async () => {
      try {
        return await SecureStore.getItemAsync(FT_AUTH_TOKEN_KEY);
      } catch {
        return storage.getItem(FT_AUTH_TOKEN_KEY);
      }
    },
    set: async (value) => {
      if (!value) {
        try {
          await SecureStore.deleteItemAsync(FT_AUTH_TOKEN_KEY);
        } catch {
          // fall through
        }
        await storage.removeItem(FT_AUTH_TOKEN_KEY);
        return;
      }

      try {
        await SecureStore.setItemAsync(FT_AUTH_TOKEN_KEY, value);
      } catch {
        await storage.setItem(FT_AUTH_TOKEN_KEY, value);
      }
    },
  };
}

export async function getFtAuth(
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<FtAuthRecord | null> {
  return getFtAuthFromStores(createTokenStore(storage), storage);
}

export async function setFtAuth(
```

```
  auth: FtAuthRecord,
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<void> {
  await setFtAuthToStores(auth, createTokenStore(storage), storage);
}

export async function clearFtAuth(
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<void> {
  await clearFtAuthFromStores(createTokenStore(storage), storage);
}

export { FT_AUTH_PROFILE_KEY, FT_AUTH_TOKEN_KEY };
```

```
// ===== File: src/lib/auth/storageCore.ts =====
import type { KeyValueStorage } from '@/lib/storage/asyncStorage';

import { isAuthExpired, parseFtAuthProfile, type FtAuthRecord } from './types';

export type TokenStore = {
  get: () => Promise<string | null>;
  set: (value: string | null) => Promise<void>;
};

export async function getFtAuthFromStores(
  tokenStore: TokenStore,
  profileStorage: KeyValueStorage,
): Promise<FtAuthRecord | null> {
  const [token, profileRaw] = await Promise.all([
    tokenStore.get(),
    profileStorage.getItem('ft_auth_profile'),
  ]);

  const profile = parseFtAuthProfile(profileRaw);
  if (!profile && !token) {
    return null;
  }

  const auth: FtAuthRecord = {
    ...profile,
    token: token ?? undefined,
  };

  if (isAuthExpired(auth)) {
    await clearFtAuthFromStores(tokenStore, profileStorage);
    return null;
  }

  return auth;
}

export async function setFtAuthToStores(
  auth: FtAuthRecord,
  tokenStore: TokenStore,
  profileStorage: KeyValueStorage,
): Promise<void> {
  const { token, ...profile } = auth;

  await Promise.all([
    tokenStore.set(token ?? null),
    profileStorage.setItem('ft_auth_profile', JSON.stringify(profile)),
  ]);
```

```
}

export async function clearFtAuthFromStores(
  tokenStore: TokenStore,
  profileStorage: KeyValueStorage,
): Promise<void> {
  await Promise.all([tokenStore.set(null), profileStorage.removeItem('ft_auth_profile')]);
}



// ===== File: src/lib/device/deviceId.ts =====
import { defaultAsyncStorage, type KeyValueStorage } from '@/lib/storage/asyncStorage';

const DEVICE_ID_KEY = 'funtalk-device-id';

function randomHex(length: number): string {
  let value = '';
  for (let index = 0; index < length; index += 1) {
    value += Math.floor(Math.random() * 16).toString(16);
  }
  return value;
}

function createDeviceId(): string {
  return [
    Date.now().toString(16),
    randomHex(8),
    randomHex(8),
    randomHex(8),
  ].join('-');
}

export async function getDeviceID(
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<string> {
  const existing = await storage.getItem(DEVICE_ID_KEY);
  if (existing) {
    return existing;
  }

  const deviceId = createDeviceId();
  await storage.setItem(DEVICE_ID_KEY, deviceId);
  return deviceId;
}



// ===== File: src/lib/env.ts =====
import Constants from 'expo-constants';
import * as Device from 'expo-device';
import { Platform } from 'react-native';

import { rewriteHostForRuntime } from '@/lib/devHost';

const DEFAULT_API_HOST = 'http://localhost:9000';
const DEFAULT_WEB_BASE_URL = 'http://localhost:19001';
const DEFAULT_OSS_BASE_URL = 'https://fun-talk-file.oss-cn-beijing.aliyuncs.com';
const DEFAULT_WECHAT_APP_ID = 'wx4f3da33035cc1d14';
const DEFAULT_WECHAT_UNIVERSAL_LINK = 'https://ai-fun-talk.com/app/';

type PublicEnvKey =
  | 'EXPO_PUBLIC_API_HOST'
  | 'EXPO_PUBLIC_WEB_BASE_URL'
```

```
  | 'EXPO_PUBLIC_ASSET_BASE_URL'
  | 'EXPO_PUBLIC_OSS_BASE_URL'
  | 'EXPO_PUBLIC_WECHAT_APP_ID'
  | 'EXPO_PUBLIC_WECHAT_UNIVERSAL_LINK';

function readProcessEnv(key: PublicEnvKey): string | undefined {
  switch (key) {
    case 'EXPO_PUBLIC_API_HOST':
      return process.env.EXPO_PUBLIC_API_HOST;
    case 'EXPO_PUBLIC_WEB_BASE_URL':
      return process.env.EXPO_PUBLIC_WEB_BASE_URL;
    case 'EXPO_PUBLIC_ASSET_BASE_URL':
      return process.env.EXPO_PUBLIC_ASSET_BASE_URL;
    case 'EXPO_PUBLIC_OSS_BASE_URL':
      return process.env.EXPO_PUBLIC_OSS_BASE_URL;
    case 'EXPO_PUBLIC_WECHAT_APP_ID':
      return process.env.EXPO_PUBLIC_WECHAT_APP_ID;
    case 'EXPO_PUBLIC_WECHAT_UNIVERSAL_LINK':
      return process.env.EXPO_PUBLIC_WECHAT_UNIVERSAL_LINK;
  }
}

function readPublicEnv(key: PublicEnvKey): string | undefined {
  const fromProcess = readProcessEnv(key);
  if (fromProcess) {
    return fromProcess;
  }

  const extra = Constants.expoConfig?.extra;
  if (extra && typeof extra[key] === 'string') {
    return extra[key];
  }

  return undefined;
}

function resolveRuntimeHost(configured: string | undefined, fallback: string): string {
  const host = configured ?? fallback;
  return rewriteHostForRuntime(host, Platform.OS, Device.isDevice);
}

export function getApiHost(): string {
  return resolveRuntimeHost(readPublicEnv('EXPO_PUBLIC_API_HOST'), DEFAULT_API_HOST);
}

export function getWebBaseUrl(): string {
  return resolveRuntimeHost(readPublicEnv('EXPO_PUBLIC_WEB_BASE_URL'), DEFAULT_WEB_BASE_URL);
}

export function getAssetBaseUrl(): string {
  return readPublicEnv('EXPO_PUBLIC_ASSET_BASE_URL') ?? getWebBaseUrl();
}

export function getOssBaseUrl(): string {
  return readPublicEnv('EXPO_PUBLIC_OSS_BASE_URL') ?? DEFAULT_OSS_BASE_URL;
}

export function getWechatAppId(): string {
  return readPublicEnv('EXPO_PUBLIC_WECHAT_APP_ID') ?? DEFAULT_WECHAT_APP_ID;
}

export function getWechatUniversalLink(): string {
  return readPublicEnv('EXPO_PUBLIC_WECHAT_UNIVERSAL_LINK') ?? DEFAULT_WECHAT_UNIVERSAL_LINK;
}
```

```
// ===== File: src/shared/courseHomeMap.ts =====
export const MAP_WIDTH = 3325;
export const MAP_SEGMENT_HEIGHT = 2155;
export const PUBLISHED_LESSON_STATUS = 1;

export type CourseHomeLesson = {
  id: number;
  lesson_key?: string;
  title?: string;
  description?: string;
  status?: number;
};

export type CourseMapNode = {
  number: number;
  lessonId: string;
  title: string;
  x: number;
  y: number;
};

const BASE_COURSE_POSITIONS: { x: number; y: number }[] = [
  { x: 925, y: 700 },
  { x: 1210, y: 760 },
  { x: 1510, y: 790 },
  { x: 1810, y: 700 },
  { x: 2100, y: 610 },
  { x: 2380, y: 605 },
  { x: 2570, y: 850 },
  { x: 2540, y: 1150 },
  { x: 2390, y: 1405 },
  { x: 2115, y: 1390 },
  { x: 1905, y: 1365 },
  { x: 1570, y: 1480 },
  { x: 1320, y: 1530 },
  { x: 1040, y: 1460 },
  { x: 755, y: 1510 },
  { x: 730, y: 1800 },
  { x: 885, y: 2050 },
  { x: 1160, y: 2080 },
  { x: 1455, y: 2025 },
  { x: 1710, y: 1950 },
  { x: 2020, y: 1950 },
  { x: 2300, y: 2050 },
  { x: 2575, y: 1940 },
];

export function getPublishedLessons(lessons: CourseHomeLesson[]): CourseHomeLesson[] {
  return lessons
    .filter((lesson) => lesson.status === PUBLISHED_LESSON_STATUS)
    .sort((left, right) => left.id - right.id);
}

export function getCoursePosition(courseIndex: number): { x: number; y: number } {
  const cycle = Math.floor(courseIndex / BASE_COURSE_POSITIONS.length);
  const base = BASE_COURSE_POSITIONS[courseIndex % BASE_COURSE_POSITIONS.length];
  return {
    x: base.x,
    y: base.y + cycle * MAP_SEGMENT_HEIGHT,
  };
}
```

```
export function getCourseMapHeight(totalCourses: number): number {
  if (totalCourses <= 0) {
    return MAP_SEGMENT_HEIGHT;
  }
  return Math.ceil(totalCourses / BASE_COURSE_POSITIONS.length) * MAP_SEGMENT_HEIGHT;
}

export function getCourseMapSegmentCount(totalCourses: number): number {
  if (totalCourses <= 0) {
    return 1;
  }
  return Math.ceil(totalCourses / BASE_COURSE_POSITIONS.length);
}

export function buildCourseMapNodes(
  lessons: CourseHomeLesson[],
  totalCourses: number,
): CourseMapNode[] {
  return Array.from({ length: totalCourses }, (_, index) => {
    const number = index + 1;
    const position = getCoursePosition(index);
    const lesson = lessons[index];
    return {
      number,
      lessonId: String(number),
      title: lesson?.title || `课程 ${number}`,
      x: position.x,
      y: position.y,
    };
  });
}

export function getCourseButtonImage(completed: boolean): string {
  return completed
    ? '/images/home/button-green.png'
    : '/images/home/button-grey.png';
}



// ===== File: src/shared/courseHomeProgress.ts =====
import type { ApiClient } from '@/lib/api/client';
import type { KeyValueStorage } from '@/lib/storage/asyncStorage';
import { defaultAsyncStorage } from '@/lib/storage/asyncStorage';

export const COURSE_HOME_PROGRESS_KEY = 'fun-talk-course-home-progress-v1';
export const FALLBACK_TOTAL_COURSES = 23;

export type CourseProgress = {
  completedCourseNumbers: number[];
  currentCourseNumber: number;
};

type CourseProgressApiResponse = {
  completed_course_numbers?: number[];
  current_course_number?: number;
};

export function parseStoredCourseProgress(
  raw: string | null,
  total: number,
): CourseProgress {
  try {
```

```
    const parsed = JSON.parse(raw || '{}') as Partial<CourseProgress>;
    const completed = Array.from(
      new Set(
        (parsed.completedCourseNumbers || [])
          .map((value) => Number(value))
          .filter((value) => Number.isInteger(value) && value >= 1 && value <= total),
      ),
    ).sort((a, b) => a - b);
    const firstOpen = Math.min(total, completed.length + 1);
    const requestedCurrent = Number(parsed.currentCourseNumber);
    const current =
      Number.isInteger(requestedCurrent) && requestedCurrent >= 1 && requestedCurrent <= tota
        ? requestedCurrent
        : firstOpen;
    return { completedCourseNumbers: completed, currentCourseNumber: current };
  } catch {
    return { completedCourseNumbers: [], currentCourseNumber: 1 };
  }
}

export function buildCompletedCourseProgress(
  courseNumber: number,
  total = FALLBACK_TOTAL_COURSES,
  previous?: CourseProgress,
): CourseProgress {
  const progress = previous ?? parseStoredCourseProgress(null, total);
  const completedCourseNumbers = Array.from(
    new Set([...progress.completedCourseNumbers, courseNumber]),
  )
    .filter((value) => value >= 1 && value <= total)
    .sort((a, b) => a - b);
  const nextCourse = Math.min(total, Math.max(courseNumber + 1, completedCourseNumbers.length
  return {
    completedCourseNumbers,
    currentCourseNumber: nextCourse,
  };
}

export function normalizeServerProgress(
  payload: CourseProgressApiResponse,
  total: number,
): CourseProgress {
  const completed = Array.from(
    new Set(
      (payload.completed_course_numbers || [])
        .map((value) => Number(value))
        .filter((value) => Number.isInteger(value) && value >= 1 && value <= total),
    ),
  ).sort((a, b) => a - b);
  const requestedCurrent = Number(payload.current_course_number);
  const current =
    Number.isInteger(requestedCurrent) && requestedCurrent >= 1 && requestedCurrent <= total
      ? requestedCurrent
      : Math.min(total, completed.length + 1);
  return { completedCourseNumbers: completed, currentCourseNumber: current };
}

export async function readCourseProgress(
  total: number,
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<CourseProgress> {
  const raw = await storage.getItem(COURSE_HOME_PROGRESS_KEY);
  return parseStoredCourseProgress(raw, total);
}
```

```
export async function writeCourseProgress(
  progress: CourseProgress,
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<void> {
  await storage.setItem(COURSE_HOME_PROGRESS_KEY, JSON.stringify(progress));
}

export async function markCourseHomeCourseCompleted(
  courseNumber: number,
  total = FALLBACK_TOTAL_COURSES,
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<CourseProgress> {
  const previous = await readCourseProgress(total, storage);
  const progress = buildCompletedCourseProgress(courseNumber, total, previous);
  await writeCourseProgress(progress, storage);
  return progress;
}

export async function fetchCourseHomeProgress(
  total: number,
  apiClient: ApiClient,
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<CourseProgress> {
  const params = new URLSearchParams({ total_courses: String(total) });
  const response = await apiClient.get(`/api/v1/course_progress?${params.toString()}`);
  if (!response.ok) {
    throw new Error(`load course progress failed: ${response.status}`);
  }

  const progress = normalizeServerProgress(
    (await response.json()) as CourseProgressApiResponse,
    total,
  );
  await writeCourseProgress(progress, storage);
  return progress;
}

export async function saveCourseHomeCourseCompleted(
  options: {
    courseNumber: number;
    lessonId: string;
    totalCourses: number;
  },
  apiClient: ApiClient,
  storage: KeyValueStorage = defaultAsyncStorage,
): Promise<CourseProgress> {
  const response = await apiClient.post('/api/v1/course_progress/complete', {
    course_number: options.courseNumber,
    lesson_id: options.lessonId,
    total_courses: options.totalCourses,
    status: 'completed',
  });

  if (!response.ok) {
    throw new Error(`save course progress failed: ${response.status}`);
  }

  const progress = normalizeServerProgress(
    (await response.json()) as CourseProgressApiResponse,
    options.totalCourses,
  );
  await writeCourseProgress(progress, storage);
  return progress;
```

```
}



// ===== File: src/shared/courseOpeningSceneEntry.ts =====
import type { ApiClient } from '@/lib/api/client';

type OpeningSceneConfigResponse = {
  success?: boolean;
  data?: Record<string, unknown> | null;
};

type LessonDetailResponse = {
  lesson?: {
    id?: number;
    source_course_id?: number | null;
  };
};

const DEFAULT_OPENING_SCENE_LOOKUP_TIMEOUT_MS = 3_500;

function withTimeout<T>(
  promise: Promise<T>,
  timeoutMs: number,
  timeoutValue: T,
): Promise<T> {
  let timeoutId: ReturnType<typeof setTimeout> | null = null;

  const timeout = new Promise<T>((resolve) => {
    timeoutId = setTimeout(() => {
      resolve(timeoutValue);
    }, timeoutMs);
  });

  return Promise.race([promise, timeout]).finally(() => {
    if (timeoutId) {
      clearTimeout(timeoutId);
    }
  });
}

export function sanitizeInternalReturnPath(path: string | null | undefined): string | null {
  if (!path || !path.startsWith('/') || path.startsWith('//')) {
    return null;
  }

  try {
    const url = new URL(path, 'http://localhost');
    return `${url.pathname}${url.search}${url.hash}`;
  } catch {
    return null;
  }
}

export function buildOpeningSceneEntryPath(lessonId: string, returnTo: string): string {
  return `/demos/openingScene?lesson_id=${encodeURIComponent(lessonId)}&return_to=${encodeURI
}

export function resolveOpeningSceneExitPath(
  lessonId: string,
  returnTo: string | null | undefined,
): string {
  const safeReturnTo = sanitizeInternalReturnPath(returnTo);
  if (safeReturnTo) {
```

```
    return safeReturnTo;
  }
  return `/demos/realtimeConversationV2?lesson_id=${encodeURIComponent(lessonId)}`;
}

export async function lessonHasOpeningSceneConfig(
  lessonId: string,
  apiClient: ApiClient,
): Promise<boolean> {
  const lessonResponse = await apiClient.get(`/api/v1/realtime_lessons/${lessonId}`);
  if (!lessonResponse.ok) {
    return false;
  }

  const lessonPayload = (await lessonResponse.json()) as LessonDetailResponse;
  const lesson = lessonPayload.lesson;
  if (!lesson) {
    return false;
  }

  const courseId =
    lesson.source_course_id && lesson.source_course_id > 0
      ? lesson.source_course_id
      : lesson.id;
  if (!courseId || courseId <= 0) {
    return false;
  }

  const configResponse = await apiClient.get(`/api/v1/opening_scene_config/${courseId}`);
  if (!configResponse.ok) {
    return false;
  }

  const configPayload = (await configResponse.json()) as OpeningSceneConfigResponse;
  return Boolean(configPayload.success && configPayload.data);
}

export async function resolveCourseLessonEntryPath(
  lessonSearch: URLSearchParams,
  skipOpeningScene: boolean,
  apiClient: ApiClient,
  openingSceneLookupTimeoutMs = DEFAULT_OPENING_SCENE_LOOKUP_TIMEOUT_MS,
): Promise<string> {
  const lessonPath = `/app/lesson?${lessonSearch.toString()}`;
  if (skipOpeningScene) {
    return lessonPath;
  }

  const lessonId = lessonSearch.get('lesson_id')?.trim() || '';
  if (!lessonId) {
    return lessonPath;
  }

  const hasOpeningScene = await withTimeout(
    lessonHasOpeningSceneConfig(lessonId, apiClient).catch((error) => {
      console.warn('opening scene lookup failed:', error);
      return false;
    }),
    openingSceneLookupTimeoutMs,
    false,
  );
  if (!hasOpeningScene) {
    return lessonPath;
  }
```

```
  return buildOpeningSceneEntryPath(lessonId, lessonPath);
}

export function buildNativeLessonPath(lessonSearch: URLSearchParams): string {
  return `/lesson?${lessonSearch.toString()}`;
}



// ===== File: src/hooks/use-theme.ts =====
/**
 * Learn more about light and dark modes:
 * https://docs.expo.dev/guides/color-schemes/
 */

import { Colors } from '@/constants/theme';
import { useColorScheme } from '@/hooks/use-color-scheme';

export function useTheme() {
  const scheme = useColorScheme();
  const theme = scheme === 'dark' ? 'dark' : 'light';

  return Colors[theme];
}



// ===== File: src/components/OpeningAnimation.tsx =====
import { useEffect } from 'react';
import { Dimensions, StyleSheet, View } from 'react-native';
import { Image } from 'expo-image';
import Animated, {
  runOnJS,
  useAnimatedStyle,
  useSharedValue,
  withDelay,
  withTiming,
} from 'react-native-reanimated';

const { width: SCREEN_W, height: SCREEN_H } = Dimensions.get('window');
const FOX_SIZE = Math.min(220, SCREEN_W * 0.52);
const LOGO_WIDTH = Math.min(260, SCREEN_W * 0.62);
const LOGO_HEIGHT = LOGO_WIDTH * 0.38;

const foxImage = require('../../assets/images/login/fox-register.png');
const logoImage = require('../../assets/images/login/logo.png');

type Props = {
  /** Called once the exit animation is complete. */
  onFinish?: () => void;
};

export function OpeningAnimation({ onFinish }: Props) {
  const logoOpacity = useSharedValue(0);
  const logoY = useSharedValue(40);
  const logoScale = useSharedValue(0.8);

  const exitOpacity = useSharedValue(1);

  // ── kick off entrance sequence ──
  useEffect(() => {
    logoOpacity.value = withDelay(450, withTiming(1, { duration: 500 }));
    logoY.value = withDelay(450, withTiming(0, { duration: 500 }));
```

```
    logoScale.value = withDelay(500, withTiming(1, { duration: 500 }));

    // Exit: fade everything out after ~2.8s
    exitOpacity.value = withDelay(
      2800,
      withTiming(0, { duration: 500 }, (finished) => {
        if (finished && onFinish) {
          runOnJS(onFinish)();
        }
      }),
    );
  }, [logoOpacity, logoY, logoScale, exitOpacity, onFinish]);

  // ── animated styles ──
  const containerStyle = useAnimatedStyle(() => ({
    opacity: exitOpacity.value,
  }));

  const logoStyle = useAnimatedStyle(() => ({
    opacity: logoOpacity.value,
    transform: [{ translateY: logoY.value }, { scale: logoScale.value }],
  }));

  return (
    <Animated.View style={[StyleSheet.absoluteFill, styles.root, containerStyle]} pointerEven
      {/* Gradient-like background layers */}
      <View style={styles.bgBase} />
      <View style={styles.bgGlow} />

      {/* Decorative circles */}
      <View style={[styles.circle, styles.circle1]} />
      <View style={[styles.circle, styles.circle2]} />
      <View style={[styles.circle, styles.circle3]} />

      {/* Main content */}
      <View style={styles.content}>
        <View>
          <Image
            source={foxImage}
            style={{ width: FOX_SIZE, height: FOX_SIZE * 1.05 }}
            contentFit="contain"
          />
        </View>

        <Animated.View style={[styles.logoWrap, logoStyle]}>
          <Image
            source={logoImage}
            style={{ width: LOGO_WIDTH, height: LOGO_HEIGHT }}
            contentFit="contain"
          />
        </Animated.View>
      </View>
    </Animated.View>
  );
}

const styles = StyleSheet.create({
  root: {
    zIndex: 999,
    elevation: 30,
  },
  bgBase: {
    ...StyleSheet.absoluteFillObject,
    backgroundColor: '#FF8C42',
```

```
  },
  bgGlow: {
    ...StyleSheet.absoluteFillObject,
    backgroundColor: '#FFD166',
    opacity: 0.45,
    // Radial glow effect via large blurred circle would need native,
    // so we use a semi-transparent overlay for warmth
  },
  circle: {
    position: 'absolute',
    borderRadius: 999,
    opacity: 0.15,
  },
  circle1: {
    width: 300,
    height: 300,
    top: -60,
    right: -80,
    backgroundColor: '#fff',
  },
  circle2: {
    width: 200,
    height: 200,
    bottom: 80,
    left: -60,
    backgroundColor: '#FFD166',
  },
  circle3: {
    width: 140,
    height: 140,
    top: SCREEN_H * 0.35,
    right: -30,
    backgroundColor: '#fff',
    opacity: 0.1,
  },
  content: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    paddingBottom: SCREEN_H * 0.06,
  },
  logoWrap: {
    marginTop: -6,
  },
});



// ===== File: src/components/themed-text.tsx =====
import { Platform, StyleSheet, Text, type TextProps } from 'react-native';

import { Fonts, ThemeColor } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

export type ThemedTextProps = TextProps & {
  type?: 'default' | 'title' | 'small' | 'smallBold' | 'subtitle' | 'link' | 'linkPrimary' |
  themeColor?: ThemeColor;
};

export function ThemedText({ style, type = 'default', themeColor, ...rest }: ThemedTextProps)
  const theme = useTheme();

  return (
    <Text
```

```
      style={[
        { color: theme[themeColor ?? 'text'] },
        type === 'default' && styles.default,
        type === 'title' && styles.title,
        type === 'small' && styles.small,
        type === 'smallBold' && styles.smallBold,
        type === 'subtitle' && styles.subtitle,
        type === 'link' && styles.link,
        type === 'linkPrimary' && styles.linkPrimary,
        type === 'code' && styles.code,
        style,
      ]}
      {...rest}
    />
  );
}

const styles = StyleSheet.create({
  small: {
    fontSize: 14,
    lineHeight: 20,
    fontWeight: 500,
  },
  smallBold: {
    fontSize: 14,
    lineHeight: 20,
    fontWeight: 700,
  },
  default: {
    fontSize: 16,
    lineHeight: 24,
    fontWeight: 500,
  },
  title: {
    fontSize: 48,
    fontWeight: 600,
    lineHeight: 52,
  },
  subtitle: {
    fontSize: 32,
    lineHeight: 44,
    fontWeight: 600,
  },
  link: {
    lineHeight: 30,
    fontSize: 14,
  },
  linkPrimary: {
    lineHeight: 30,
    fontSize: 14,
    color: '#3c87f7',
  },
  code: {
    fontFamily: Fonts.mono,
    fontWeight: Platform.select({ android: 700 }) ?? 500,
    fontSize: 12,
  },
});



// ===== File: src/components/themed-view.tsx =====
import { View, type ViewProps } from 'react-native';
```

```
import { ThemeColor } from '@/constants/theme';
import { useTheme } from '@/hooks/use-theme';

export type ThemedViewProps = ViewProps & {
  lightColor?: string;
  darkColor?: string;
  type?: ThemeColor;
};

export function ThemedView({ style, lightColor, darkColor, type, ...otherProps }: ThemedViewP
  const theme = useTheme();

  return <View style={[{ backgroundColor: theme[type ?? 'background'] }, style]} {...otherPro
}
```